Neural Computing and Applications (2024) 36:20295–20320
https://doi.org/10.1007/s00521-024-10237-8

ORIGINAL ARTICLE

Enhancing dynamic ensemble selection: combining self-generating prototypes and meta-classifier for data classification

Alberto Manastarla[1] · Leandro A. Silva[2]

Received: 20 February 2024 / Accepted: 12 July 2024 / Published online: 12 August 2024
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2024

Abstract

In dynamic ensemble selection (DES) techniques, the competence level of each classifier is estimated from a pool of classifiers, and only the most competent ones are selected to classify a specific test sample and predict its class labels. A significant challenge in DES is efficiently estimating classifier competence for accurate prediction, especially when these techniques employ the K-Nearest Neighbors (KNN) algorithm to define the competence region of a test sample based on a validation set (known as the dynamic selection dataset or DSEL). This challenge is exacerbated when the DSEL does not accurately reflect the original data distribution or contains noisy data. Such conditions can reduce the precision of the system, induce unexpected behaviors, and compromise stability. To address these issues, this paper introduces the self-generating prototype ensemble selection (SGP.DES) framework, which combines meta-learning with prototype selection. The proposed meta-classifier of SGP.DES supports multiple classification algorithms and utilizes meta-features from prototypes derived from the original training set, enhancing the selection of the best classifiers for a test sample. The method improves the efficiency of KNN in defining competence regions by generating a reduced and noise-free DSEL set that preserves the original data distribution. Furthermore, the SGP.DES framework facilitates tailored optimization for specific classification challenges through the use of hyperparameters that control prototype selection and the meta-classifier operation mode to select the most appropriate classification algorithm for dynamic selection. Empirical evaluations of twenty-four classification problems have demonstrated that SGP.DES outperforms state-of-the-art DES methods as well as traditional single-model and ensemble methods in terms of accuracy, confirming its effectiveness across a wide range of classification contexts.

Keywords Prototype selection · Data reduction · Dynamic ensemble selection · Meta-learning

1 Introduction

Multiple classifier systems (MCS) have emerged as a significant area of research in recent decades, presenting innovative and effective solutions for complex classification challenges [1]. These methods exhibit superior performance compared to individual classifiers, as supported by various studies [1–3]. This superiority is particularly evident in dynamic and nonstationary environments where continuous adaptation is essential for maintaining high classification accuracy [4–6]. MCS comprises three stages [7]: generation, selection, and integration. Initially, a pool of classifiers is generated (generation phase). In the selection phase, either a single classifier or a subset from the pool, known as the ensemble of classifiers (EoC), is selected. The final stage, integration phase, also known as

✉ Alberto Manastarla
10343964@mackenzista.com.br

Leandro A. Silva
leandroaugusto.silva@mackenzie.br

[1] Graduate Program in Electrical Engineering and Computing, Mackenzie Presbyterian University, R. da Consolação, 930 - Consolação, Sao Paulo, SP 01302-907, Brazil

[2] Computing and Informatics Faculty and Graduate Program in Electrical Engineering and Computing, Mackenzie Presbyterian University, R. da Consolação, 930 - Consolação, Sao Paulo, SP 01302-907, Brazil

the generalization phase, involves amalgamating the predictions of these chosen classifiers to reach a decision—data classification.

Classifier selection can be static or dynamic. Static selection involves well-established methods such as Bagging, Boosting, and Stacking, utilizing various strategies during training to form EoC based on performance criteria [8–11]. Conversely, in dynamic selection (DS), the EoC is determined during the generalization phase. Thus, for each test sample, the base classifiers competent is assessed using a selection criterion, and only the most competent classifiers are selected for prediction [6]. Dynamic selection, on the other hand, can be divided into two categories: dynamic classifier selection (DCS) and dynamic ensemble selection (DES). Basically, DCS selects a single classifier, while DES selects a subset of classifiers and combines their outputs [6].

However, recent studies have indicated that DES methods often yield better classification accuracy than DCS methods across various problem domains, such as imbalanced datasets and concept drift scenarios, which are crucial in real-time machine learning applications [12–20].

The DES methods are particularly effective because they recognize that different classifiers may exhibit expertise—or specialization—in various local regions of the feature space, which are referred to as competence regions. This specialization underscores the strategy of selecting the most promising classifiers for analyzing a specific instance. The competence of classifiers is typically estimated based on a defined criterion, calculated within the neighborhood (competence region) of the instance under consideration. This calculation is performed using a validation data, also known as the dynamic selection dataset (DSEL), which helps in identifying the most competent classifiers for the given instance.

High noise levels in the DSEL significantly challenge dynamic ensemble selection (DES) methods. This noise can cause the competence region to misrepresent the instance being analyzed, leading to the selection of classifiers that may not perform optimally. This issue is particularly severe in DES methods that rely on K-Nearest Neighbors (KNNs) to define competence regions, as noise heavily impacts KNN performance [21].

Choosing the right criterion to estimate classifier competence is another hurdle [21]. Relying on a single criterion might not accurately reflect a classifier's ability, and not customizing criteria to the specific problem may increase computational demands or lead to system inaccuracies.

Incorporating meta-learning into DESs by analyzing additional information from the instances under analysis has been shown to significantly enhance classifier selection precision, improving classifier ensemble formation in the generalization phase [21–23].

This work introduces a novel dynamic ensemble selection framework, named self-generating prototype ensemble selection (SGP.DES). This framework uniquely combines prototype selection with dynamic ensemble selection and meta-learning, selecting the most representative prototypes from the training set to form a reduced and noise-free validation set (DSEL). The selected prototypes used to create the DSEL set tend to preserve the statistical properties of the original training set. Furthermore, the SGP.DES employs meta-feature extraction from these prototypes to train a meta-classifier, which is adept at dynamically selecting the optimal base classifier(s) for each instance under analysis. This meta-classifier can operate with different classification algorithms, such as KNN, Random Forest, and SVM, to assess the competence of base classifiers in the pool based on the extracted meta-features from the selected prototypes and the instance under analysis.

The proposed SGP.DES approach enhances the efficiency of the KNN algorithm in defining the competence region for each instance during the generalization phase by using a reduced and noise-free DSEL set. Additionally, employing prototype selection to reduce the DSEL set enables the meta-classifier to be trained with relevant meta-features from representative training set instances. This streamlined process not only improves computational efficiency, but also facilitates the formation of an effective EoC for accurately predicting the class of each test instance. The adaptability of the SGP.DES, supported by multiple selection criteria and the integration of various classification algorithms in its meta-classifier, makes it particularly effective for imbalanced datasets and dynamic and nonstationary environments, which are key considerations in real-time machine learning applications.

The remainder of this paper is organized as follows: Sect. 2 provides a brief overview of the data reduction and dynamic classifier selection methods, while Sect. 3 describes the proposed framework. The methodology and experimental results are presented in Sects. 4 and 5, respectively. The final section concludes the paper with a summary of the findings and potential future directions.

2 Related works and background

2.1 Dynamic selection

In static ensemble methods such as decision forests [24] and Boosting [25], classifiers are fixed during training to predict test samples. In contrast, dynamic ensemble selection methods [7] dynamically choose a single classifier c_i or an ensemble C' for each test sample $\mathbf{x}_j$, enhancing

adaptability across various applications from credit scoring [26] to HIV-1 Env variance prediction [27].

Dynamic selection (DS) identifies the most competent classifier c^* or ensemble from C for a given test instance $\mathbf{x}_j$, considering the competence region θ_j:

$$c^* = \arg\max_{c_i \in C} \delta_{i,j}(\mathbf{x}_j, \theta_j, c_i) \quad (1)$$

The evolution of DS methods [28–30] has led to sophisticated methodologies involving local competence region definition, selection criterion establishment, and the choice between DCS or DES [6, 7].

Significant advances include DCS-LA by Woods et al. [31] and similarity definition methods by Giacinto et al. [32]. Adaptive metrics and realistic DCS-LA limits were explored by Didaci et al. [33, 34].

Further advancements in DS were marked by the attribute-oriented dynamic classifier selection (AO-DCS) method introduced by Zhu et al. [35], indicating a shift toward more integrative methods. This approach was further enhanced by Kuncheva et al. [36], who showed how including additional information from KNN could improve DS methods. The region of competence, typically defined by the nearest neighbor rule applied to either training [31] or validation data [37], serves as a basis for various competence measurement sources in the DES literature [7]. These include accuracy-based measures such as overall local accuracy (OLA) [31], local classifier accuracy (LCA) [31], and modified local accuracy (MLA) [31]; ranking information such as classifier rank [38] and simplified classifier rank [31]; probabilistic information derived from base classifiers decisions such as the Kullback–Leibler divergence, DES-KL [39], and the randomized reference classifier DES-PRC [40]; and classifier behavior analysis using methods such as KNOP [41] and the KNORA family [37] based on Oracle information. Brun et al. [42] discussed the role of data complexity measures, such as Fisher's Discriminant Ratio [43], in identifying the most competent classifiers. Additionally, some criteria assess the competence level of an entire ensemble of classifiers rather than each base classifier individually, using metrics such as the consensus degree in the dynamic overproduction and choose (DOCS) method [44], diversity [42, 45], and data handling [17].

The use of meta-features in DS (dynamic selection) and DES (dynamic ensemble selection) methods has been emphasized in various studies [22, 42, 46, 47]. Novel methods such as the HaO-DES, proposed by Cordeiro et al. [48], have shown promise. In the realm of multi-label classification, approaches such as CHADE by Pinto et al. [49] and MLDE by Zhu et al. [50] have been beneficial.

According to the literature, local competence regions can be defined using different methods, such as KNN [51], clustering methods [52], decisions of base classifiers [32] or a competence map, which is defined through the use of a potential function [40]. In all the cases, a set of labeled instances, which may originate from training or validation sets (DSEL), as described in [22, 46], is used. For calculating the competence of the classifier c_i at a point $\mathbf{x}$, the KNN algorithm is commonly used, i.e., the KNN from the DSEL. Therefore, in this case, the DSEL is defined, for example, as the neighbors of $\mathbf{x}$ in a DSEL, where the classes of the instances are known [53]. Additionally, as known in the literature, the majority of dynamic selection (DS) methods rely on the KNN concept, where the quality of the neighborhood around an instance significantly impacts DS methods using KNN.

KNN is preferred over clustering because it offers a more accurate estimation of the competence region, leading to varied EoCs configurations based on the classification of new instances [54, 55]. However, using KNN, as opposed to clustering, incurs a higher computational cost since the distance between the instance and the entire DSEL must be estimated before assessing the competence of base classifiers [29].

To address the challenges associated with the KNN algorithm in defining competence regions and optimizing the estimation of the competence of base classifiers with respect to the instances under analysis, this work introduces the dynamic selection (DS) framework, SGP.DES. This innovative framework enhances dynamic ensemble selection by merging meta-learning with meta-features from local competence regions of specific instances under analysis. It utilizes a prototype selection method named the adaptive self-generating prototype (A-SGPE) to improve minority class selection from training data, yielding more representative prototypes and a cleaner dynamic selection dataset (DSEL).

SGP.DES has the capability to estimate the competence of classifiers, thereby forming an effective EoC for each instance analyzed. This can be achieved by the embedded meta-classifier in SGP.DES, which can operate with different classification algorithms based on optimized models or instance-based approaches, ensuring adaptability to various classification challenges. This approach gives SGP.DES versatility for dynamic selection.

Another distinctive feature of SGP.DES is its ability to extract meta-features from selected training instances (prototypes), thereby refining the prototype selection process to create a high-quality DSEL set. This approach can potentially improve classifier selection and enhance the effectiveness of the EoC dynamically created for a specific test sample under analysis, enhancing the generalization performance of the SGP.DES better reflects to Oracle performance in terms of accuracy.

2.2 Prototype selection applied to dynamic selection

Prototype selection (PS) in machine learning is an important technique for reducing the training set size while aiming to maintain or enhance model accuracy by choosing a representative subset, termed the prototype set (P), to replace the original training set (T).

The prototype selection problem is mathematically defined as follows:

$$P^* = \arg\min_{P \subseteq TR} |P| \quad \text{subject to} \quad E(P) \leq E(T) \tag{2}$$

where P^* is the optimal prototype set, $|P|$ is the size of P, $E(P)$ is the classification error using P, and $E(T)$ is the error using the original set T. The goal is to find the smallest P^* with a classification error equal to or lower than T.

In dynamic selection, instance reduction aims to minimize or eliminate instances adversely affecting DS methods, reducing computational demands and data complexity [56]. These methods fall into two categories: instance selection [57] and prototype generation [58]. Instance selection methods, or prototype selection (PS), are particularly effective for computational efficiency and noise tolerance. These methods can be categorized into condensation, editing, and hybrid methods.

Research has shown that prototype selection (PS) methods can improve the accuracy of dynamic ensemble selection (DES) systems, in which classifiers are dynamically chosen for each test instance [59]. The composition of the dynamic selection dataset (DSEL) significantly influences the performance of dynamic selection (DS), with PS methods potentially enhancing accuracy while reducing execution time [30]. Moreover, editing methods such as random mutation hill climbing (RMHC) [60], relative neighborhood graph (RNG) [61], and edited nearest neighbor (ENN) [62] have demonstrated promising results in minimizing the DSEL size and enhancing the accuracy of DS methods.

The review by [29] addressed prototype selection in dynamic classifier and ensemble selection, highlighting the need for future research to have focused on improving classification accuracy by removing DSEL data instances. Methods that utilized local accuracy metrics, such as overall local accuracy (OLA) and local class accuracy (LCA), were shown to be particularly effective. Furthermore, Instance Hardness sampling was proposed as an alternative for noise management to ENN, maintaining the effectiveness of dynamic selection (DS) methods in noisy scenarios, as investigated by [63].

These findings underscore the significance of prototype selection in enhancing dynamic selection (DS) methods. They suggest that removing noise from the dynamic selection dataset (DSEL) results in superior performance of DS methods in complex scenarios, compared to when prototype selection (PS) methods are not used.

3 The SGP.DES framework

An overview of the self-generating prototype dynamic ensemble selection (SGP.DES) architecture is depicted in Fig. 1. This diagram illustrates the two operational modes of the embedded meta-classifier within the proposed framework. The first mode, the model-based mode, employs model-based learning where meta-features extracted from selected prototypes of the original training set are used to train the meta-classifier. This training enables the meta-classifier to predict the most effective base classifiers for the final ensemble during the generalization phase. The second mode, the instance-based mode, also utilizes meta-features extracted from selected instances of the original training data. In this mode, these features act as a model to guide the meta-classifier in predicting the optimal base classifiers for the final ensemble in the generalization phase.

The architecture is structured into three distinct phases: (1) generation, (2) meta-training, and (3) generalization, with the first two phases, the generation and the meta-training, conducted offline.

In the generation phase, a pool of base classifiers is created, and prototypes are selected to form the validation set, known as the DSEL.

During the meta-training phase, meta-features are extracted from the DSEL set to form the DSEL* set, and the meta-classifier (denoted as λ) is configured. When the hyperparameter M_λ is set to model-based, the DSEL* set is used to create a predictive model for the meta-classifier to predict the most competent base classifier during the generalization phase. Conversely, when M_λ is set to instance-based, only meta-feature extraction occurs during the Meta-training phase, resulting in the DSEL* set being used directly as a model for the meta-classifier during the generalization phase.

The generalization phase is executed on-the-fly and is dedicated to predicting the class of unseen instances by dynamically selecting the most competent base classifiers to form the final ensemble. This phase also involves an integration process where the predictions from the selected base classifiers are combined using a weighted approach, resulting in the final decision regarding the class of the instance under analysis.

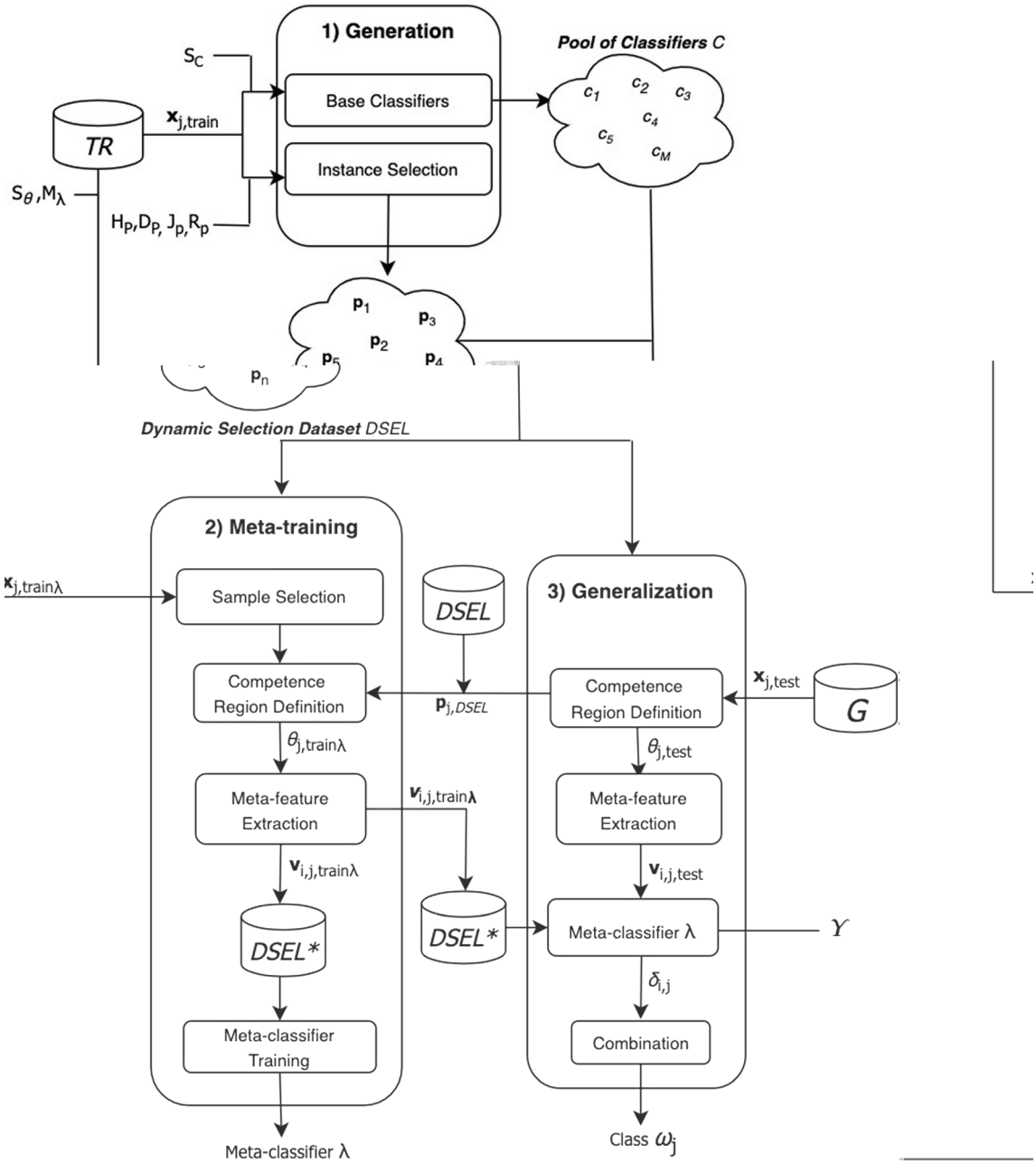

Fig. 1 Overview of the proposed framework. The process is divided into three steps: (1) Generation, where the pool of classifiers $C = \{c_1, \ldots, c_M\}$ is generated, and the most representative prototypes $\mathbf{p}_1$, $\mathbf{p}_2$, …, $\mathbf{p}_n$ from the original training set are selected to form the dynamic selection dataset (DSEL) through the instance selection process; (2) Meta-training, the training of the selector λ (meta-classifier); and (3) Generalization , where the competence level δ_i of each base classifier c_i is specifically calculated for each new test sample $\mathbf{x}_j$.S_θ, S_c, M_λ, H_p, D_p, J_p, and R_p are the hyperparameters required by the proposed system

3.1 Generation phase

The generation phase encompasses two principal processes:

- Selection of prototypes from the training set to form the dynamic selection dataset (DSEL) and extraction of meta-features from DSEL, based on the competence region of the instance under analysis, to form the *DSEL* *. This set assists in optimizing the meta-classifier during the meta-training phase or acts as a model for the *instance-based* mode of the meta-classifier during the generalization phase.
- Similarly to [22, 46], the pool of classifiers is created using the Bagging method [8]. The Bagging method works by randomly selecting different bootstraps of the

data for training each base classifier c_i. Each bootstrap uses 50% of the training data.

The pool of base classifiers C in the framework consists of perceptrons for binary and multi-class classification problems, forming a homogeneous pool of classifiers to be trained and evaluated for dynamic selection. The choice to use perceptrons is based on their efficiency in nonlinear problems, as reported in [64]. Linear classifiers, such as perceptrons, are favored for their ability to address complex nonlinear problems using linear approaches, thereby requiring less computational demand and training data to create the initial pool of classifiers [22, 46].

3.1.1 Instance selection step

For instance selection, the adaptive self-generating prototype entropy (A-SGPE) method is proposed. This method improves upon the original self-generating prototype entropy (SGPE) [65] by ensuring representativeness and balanced representation of minority classes, addressing the challenges presented by imbalanced datasets. It adapts the selection parameter K to include minority classes equitably.

A-SGPE operates through the following steps:

1. *Initialization:* Set the hyperparameters and instantiate the training set.
2. *Training set split:* Divide the training set into subsets.
3. *Entropy evaluation:* Calculate the entropy for each subset.
4. *Stopping criterion:* Cease prototype generation when the entropy variability falls below a predefined threshold.
5. *Prototype selection:* Select instances nearest to the centroids of suitable subsets to form the optimized DSEL set.

This approach creates a representative *DSEL* from *TR*, focusing on prototypes. By dividing the *TR* and evaluating the entropy of the subsets, the inclusion of minority classes is ensured, thereby improving the balance and decision-making capability of the meta-classifier.

The hyperparameters integral to the A-SGPE method include the following:

- H_P: The maximum allowed entropy threshold, limiting the entropy for the subsets to be included in the prototype set.
- J_P: The windowing period, or the size of the moving window for averaging entropy, which influences the sensitivity to entropy changes over iterations.
- D_P: The entropy level deviation threshold that determines the acceptable variability in entropy among subsets and is crucial for assessing the convergence of the prototype generation process.
- R_P: The minority class preservation rate. The selection parameter K is adjusted to ensure the balanced inclusion of minority classes in the *DSEL* set.

The prototype generation process in A-SGPE is governed by the *GeneratingPrototypes* function, which is pivotal in determining the optimal moment to halt the division of subsets and the calculation of entropy. This stop criterion ensures that the generated prototypes are representative of the original dataset, especially in terms of minority class distribution. The function is delineated as follows:

$$GeneratingPrototypes(J_P, E, D_P) = \begin{cases} \text{continue}, & \text{if } SD(J_P, E) \leq D_P \\ \text{stop}, & \text{otherwise} \end{cases} \tag{3}$$

The hyperparameters in function 3 include: the Moving Average Window Size (J_P), which influences the number of recent subsets considered for calculating the moving average of entropies and plays a critical role in the process's responsiveness to entropy fluctuations; the Entropy Set (E), which comprises the entropies determined for each subset throughout the process and is vital for assessing the diversity and representativeness of the subsets; and the entropy level deviation threshold (D_P), which establishes the standard deviation threshold for the entropy moving average and is key to determining when to end the prototype generation process.

After prototype generation, the prototype selection function, *SelectPrototype*, identifies specific instances from the original training set (*TR*) based on their proximity to the centroids of the generated subsets, as detailed in Function 4. The decision to select or discard instances is based on the efficacy (E) of each subset S_i relative to a predefined threshold H_P. When a subset S_i is considered for selection, the centroid of S_i is calculated and stored in $\mathbf{p}$, and the class ratio is determined by the ratio of the number of majority class instances to the number of minority class instances.

At this point, the CalculateK function is invoked to adjust the value of K, which determines how many instances will be selected from *TR* based on proximity to $\mathbf{p}$. The adjustment of K uses the formula $K = \lceil \max(CR \times |S_i| \times R_P, 1) \rceil$, where CR is the class ratio within subset S_i and R_P is the minority class preservation rate. This process ensures that the inclusion of instances in the *DSEL* is balanced, fostering a fair representation of minority classes. The dynamic adjustment of K facilitates a careful selection of instances that best reflect the class distribution in the original training set, promoting an effective balance between classes.

After prototype generation, the prototype selection function, *SelectPrototype*, chooses specific instances from

the original training set (TR) based on their proximity to the centroids of the generated subsets, as explicated in Function 4:

$$
\begin{aligned}
&SelectPrototype(S, TR) \\
&= \begin{cases}
\text{if } E(S_i) \leq H_P; then: & \\
\quad \text{Calculate the centroid of } S_i \text{ and store in } \mathbf{p} & \\
\quad \text{select}, & \\
\quad K \leftarrow \text{CalculateK}(S_i, R_P), & \\
\quad DSEL \leftarrow d(\mathbf{p}, TR, K), & \\
\text{discard}, \quad \text{otherwise}. &
\end{cases}
\end{aligned} \tag{4}
$$

The A-SGPE algorithm is described in Algorithm 1; the algorithm begins by initializing with the full training set (line 2), iteratively partitioning the largest subset into smaller groups (lines 7–8) and storing these subsets in the S set (line 9). The iteration stops when the variations in entropy meet a certain threshold or after a maximum number of iterations is reached (line 11). For subsets with an entropy below a certain level (line 16), centroids are calculated, and a specified number of closest instances are selected based on the class distribution and a minority preservation rate (lines 17–21), resulting in an optimized, reduced selection set (DSEL).

Algorithm 1 Adaptive self-generating prototype entropy (A-SGPE)

Input:
TR: Training dataset
J_p

3.2 Meta-training phase

During the meta-training phase, meta-features are extracted from the *DSEL* set to form the *DSEL** set. This process aims to optimize the meta-classifier based on its mode of

operation—either model-based or Instance-based. In the model-based mode, the meta-classifier is trained using the *DSEL** set. Conversely, in the Instance-based mode, the *DSEL** set is used as a model for the meta-classifier during the generalization phase.

3.2.1 Sample selection

As described in Fig. 1, the meta-training phase of the SGP.DES framework focuses on extracting meta-features from a portion of the original training set through the sample selection process. These meta-features, extracted based on this sample set, will be used throughout the subsequent competence region definition and meta-feature extraction processes to form the *DSEL** set.

3.2.2 Competence region definition

The competence regions of the instances under analysis $x_{j,train\lambda}$, which come from a portion of the original training set, are defined using the *DSEL* set through the *competence region definition* process. Instances from the *DSEL* set that form these competence regions (i.e., the neighborhoods of the instances under analysis) are selected using the KNN algorithm. The number of nearest neighbors (k) is determined by the S_θ hyperparameter of the framework.

3.2.3 Meta-feature extraction

From *DSEL* the objective of the *meta-feature extraction* process is to refine the predictive model of the meta-classifier λ. The SGP.DES framework utilizes seven meta-features, drawn from related studies [6, 7, 22, 46, 66] and supports the addition of additional meta-features to enhance decision-making. Each meta-feature (f_i) examines a particular base classifier behavior aspect, which is crucial for dynamically assessing classifier competence. This approach allows the meta-classifier to adjust and improve accuracy in various scenarios.

Base classifier evaluation is conducted within a local competence region for each analyzed instance, which is essential during both the meta-training phase, which generates the *DSEL* * set, and the generalization phase for classifying unseen instances. This evaluation covers confidence, probabilistic estimations, and accuracy, facilitating tailored classifier selection for each instance.

For any given instance $\mathbf{x}_{j,\text{train}\lambda}$, the extraction of meta-features involves identifying its local competence region $\theta_j = \mathbf{p}_1, \ldots, \mathbf{p}_n$ within the dynamic selection dataset *DSEL* using the K-Nearest Neighbors algorithm, and converting $\mathbf{x}_j$ into an output profile $\tilde{\mathbf{x}}_j = [\tilde{\mathbf{x}}_{j,1}, \tilde{\mathbf{x}}_{j,2}, \ldots, \tilde{\mathbf{x}}_{j,i}]$. Each $\tilde{\mathbf{x}}_{j,i}$ reflects the decision of the base classifier c_i for $\mathbf{x}_j$, and seven meta-features are extracted from θ_j corresponding to $\mathbf{x}_j$ to form the feature vector $\mathbf{v}_{i,j}$ labeled with the competence of the base classifier. The class labels assigned to $\mathbf{v}_{i,j,\text{train}\lambda}$ may only take the values 'competent' or 'not competent' depending on whether the base classifier accurately predicted the class label of $\mathbf{x}_{j,\text{train}\lambda}$. This vector undergoes min–max normalization to scale its values between 0 and 1, using the formula $\mathbf{v}_{i,j,\text{norm}} = \frac{\mathbf{v}_{i,j} - \min(\mathbf{v}_{i,j})}{\max(\mathbf{v}_{i,j}) - \min(\mathbf{v}_{i,j})}$, where $\mathbf{v}_{i,j,\text{norm}}$ is the normalized vector. The minima and maxima of $\mathbf{v}_{i,j}$ determine the scale. These normalized vectors compose the enhanced dataset $DSEL^*$. The meta-features extracted from this process are described in Table 1.

The accuracy metrics used are as follows:

1. General local accuracy: f_{ola} The accuracy of c_i over the entire competence region θ_j (Eq. 5).

$$f_{\text{ola}} = \frac{1}{K}\sum_{k=1}^{K} P(w_l|\mathbf{x}_k \in w_l, c_i) \tag{5}$$

2. The conditional local accuracy of c_i, f_{cond}, is estimated in relation to the output classes w_l (w_l is the class assigned to $\mathbf{x}_j$ by c_i) for the samples (prototypes) that belong to the competence region θ_j (Eq. 6).

$$f_{\text{cond}} = \frac{\sum_{\mathbf{x}_k \in w_l} P(w_l|\mathbf{x}k, c_i)}{\sum k = 1^K P(w_l|\mathbf{x}_k, c_i)} \tag{6}$$

Confidence Averages:

1. The average distance of all prototypes contained in the competence region relative to the decision boundary, $f_{dis,prototype}$, is given by the base classifier; c_i can be calculated by the following equation 7:

$$f_{dist,prototype} = \frac{1}{|\theta_i|}\sum_{\mathbf{p}_j \in \theta_i} d(\mathbf{p}_j, H_i) \tag{7}$$

where θ_i is the competence region of the classifier; c_i, $\mathbf{p}_j$ is a prototype contained in θ_i; H_i is the decision boundary of the classifier c_i; and $d(\mathbf{p}_j, H_i)$ is the distance between the prototype $\mathbf{p}_j$ and the decision boundary H_i.

2. The $f_{dist,instance}$, distance of the analyzed instance $\mathbf{x}_j$ relative to the decision boundary of the base classifier c_i, can be calculated using Eq. 8:

$$f_{dist,instance} = d(\mathbf{x}_j, H_i) \tag{8}$$

where $d(\mathbf{x}_j, H_i)$ represents the distance from instance $\mathbf{x}_j$ to the decision boundary H_i of classifier c_i.

3. The measure $f_{dist,centroid}$, given by Eq. 10, calculates the distance between the centroid of the local competence region θ and the analyzed instance $\mathbf{x}_j$. The centroid $\bar{\mathbf{p}}_\theta$

is determined by the average of the feature vectors of all the prototypes in θ, as defined in Eq. 9.

$$\bar{\mathbf{p}}_{\theta} = \frac{1}{|\theta|} \sum_{p_j \in \theta} \mathbf{p}_j \tag{9}$$

where $|\theta|$ is the number of prototypes in the competence region θ, and $\mathbf{p}_j$ is the feature vector of the j-th prototype.

The distance is then calculated as:

$$f_{dis,centorid} = \sqrt{\sum_{i=1}^{n} (\bar{\mathbf{p}}_{\theta_i} - \mathbf{x}_{ji})^2} \tag{10}$$

Here, $\bar{\mathbf{p}}_{\theta}$ is the centroid vector, $\mathbf{x}_j$ is the analyzed instance, and n is the number of features.

Probabilistic averages:

1. The measure $f_{overall,support}$, Eq. 11, calculates the support class for a local competence region θ of a classifier c_i:

$$f_{overall,support} = *arg\, max_{\omega_l} P(\omega_l | \mathbf{x}_j, c_i) \tag{11}$$

With S_l the function that calculates the support of the local competence region θ, $*arg, max_{\omega_l}$ is the argument that maximizes the function, ω_l are the possible output classes, and $P(\omega_l | \mathbf{x}_j, c_i)$ is the conditional probability of the output class being ω_l, given the instance $\mathbf{x}_j$ and the classifier base c_i.

2. The measure $f_{cond,support}$, which provides local conditional region support to the class predicted by the classifier c_i for the analyzed instance $\mathbf{x}_j$, is given by Eq. 12:

$$f_{cond,support} = P(\hat{\omega} | \mathbf{x}_j, c_i) \tag{12}$$

Where S is the function that calculates the support of the local competence region θ, $\hat{\omega}$ is the class predicted by the classifier c_i, and $P(\hat{\omega} | \mathbf{x}_j, c_i)$ is the conditional probability of the predicted class $\hat{\omega}$, given the instance $\mathbf{x}_j$ and the base classifier c_i.

3.3 Meta-classifier training

During the meta-training phase, meta-feature is extracted from the *DSEL* set to form the *DSEL** set. This process aims to optimize the meta-classifier based on its mode of operation—either model-based or instance-based. In the model-based mode, the meta-classifier is trained using the *DSEL** set. Conversely, in the Instance-based mode, the *DSEL** set is used as a model for the meta-classifier during the generalization phase.

As described in Fig. 1, the meta-training phase of the SGP.DES framework focuses on extracting meta-features from a portion of the original training set through the *sample selection* process. This specifically targets defining the competence region of that portion of instances through the *competence region definition* process, which uses the dynamic selection set, *DSEL*, to extract meta-features by the *meta-feature extraction* process to refine the predictive model of the meta-classifier λ. During this phase, the extracted meta-features serve a dual role: they function as a combined training and validation set (e.g., in cross-validation) for the model-based mode of the meta-classifier, or as the predictive model itself (*DSEL**) in its instance-based mode. The meta-features are amalgamated from two distinct datasets: instances from the original set *TR*, represented by $TR_{\lambda} = \{\mathbf{x}_{1,\text{train}\lambda}, \ldots, \mathbf{x}_{N,\text{train}\lambda}\}$ with N indicating the number of instances, set to a predetermined proportion of 25% of *TR* for training the meta-classifier, and *DSEL*. The derived meta-feature, *DSEL**, is utilized to train the model in the model-based mode, or to serve as the predictive model in the instance-based mode.

The algorithm, referred to as Phase 2: Meta-training (Algorithm 2), is designed to prepare a meta-classifier (lines 1-12). It operates with a dynamic selection set (*DSEL*) and a subset of training data (TR_{λ}) and utilizes a specified operating mode for the meta-classifier (M_{λ}), which can be either model-based or instance-based.

The procedure starts by initializing an enhanced metadata set (*DSEL**) (line 1). The core of the algorithm involves a nested loop where meta-feature is extracted for

Table 1 Meta-Features in the SGP.DES framework

Meta-feature	Domain	Description
f_{ola}	Accuracy	Overall local accuracy
f_{cond}	Accuracy	Conditional local accuracy
$f_{dist,prototype}$	Confidence	Distance of prototypes to decision boundaries
$f_{dist,instance}$	Confidence	Distance of instance to decision boundaries
$f_{support,overall}$	Probabilistic	Overall support of the competence region
$f_{support,cond}$	Probabilistic	Conditional support of the competence region
$f_{dist,centroid}$	Confidence	Distance from centroid to instance

assigned based on the level of support or confidence each classifier contributes, ensuring a robust and well-informed classification outcome.

Algorithm 3 Phase 3: Generalization phase

Input:
$\mathbf{x}_{j,test}$: Test instance
DSEL: Dynamic Selection Dataset
$DSEL^*$: Meta-feature Dynamic
S_θ: Size of the Competence Region
Meta-classifier λ
Output: Predicted Class Label of the test instance, ω_j
1: Find k closest prototypes in *DSEL* to $\mathbf{x}_{j,test}$ ▷ Define the competence region for $\mathbf{x}_{j,test}$ based on the S_θ hyperparameter defined in Meta-Training phase
2: Extract and normalize meta-features from $\theta_{j,test}$ of $\mathbf{x}_{j,test}$ for each c_i in C.
3: **if** mode == 'Instance-Based' **then**
4: Calculate distances between extracted $\mathbf{v}_{i,j,test}$ and $DSEL^*$ instances
5: Select only the base classifiers that labeled $\delta_{i,j}$ as 'competent' based on the closest distances between $\mathbf{v}_{i,j,test}$ and $\mathbf{v}_{i,j,training\lambda}$ in $DSEL^*$
6: **else**
7: Select only the base classifiers labeled as 'competent' according to the predictions of λ based on $\mathbf{v}_{i,j,test}$ vectors for each c_i in C
8: **end if**
9: Perform weighted majority voting with competent classifiers

4 Experimental methodology

The experimental evaluation of the proposed approach involved the analysis of twenty-four well-known datasets from the KEEL [67] and UCI machine learning repositories [68], as specified in Table 2. These datasets are chosen for their diverse characteristics to demonstrate the general applicability and robustness of the proposed approach.

Each dataset was subjected to twenty repetitions of experiments. For each repetition, the datasets were divided using the holdout method, in which 50% of the data were allocated for training, 25% were allocated for *DSEL*, and 25% were allocated for testing while maintaining class prior probabilities.

For binary classification problems, the pool of classifiers consisted of 50 perceptrons generated using the Bagging method. For multi-class problems, 50 multi-class perceptrons were used. The selection of perceptron classifiers was justified by previous works [22, 46], which demonstrated their effectiveness in solving complex nonlinear classification problems.

The following procedure outlines the steps to determine the optimal SGP.DES hyperparameters (J_p^*, D_p^*, H_p^*, R_p^* and M_λ^*) for each of the twenty-four datasets listed in Table 2 for each round of validation:

(1) For each combination of J_p, D_p, H_p R_p and M_λ as specified in Table 3 for SGP.DES.
(2) Apply the SGP.DES algorithm with the current combination of hyperparameters.
(3) Performance was evaluated on the training set.
(4) Record the combination of hyperparameters if it yields the best performance.
(5) End For
(6) Define J_p^*, D_p^*, H_p^*, R_p^* and M_λ^* as the combinations that achieved the best performance.

The proposed approach was compared with dynamic classifier selection (DCS), dynamic ensemble selection (DES) and static ensemble methods. These include the SINGLE-BEST, STACKED, STATIC-SELECTION, LCA, MCB, MLA, OLA, DCS-RANK, DCS-APRIORI, OLA, LCA, KNORA-E, KNORA-U, META-DES, Bagging, Adaboost, Random Forest, 1NN, and SVM with Radial Kernel, as detailed in previous studies [22, 46, 51].

Classification accuracies are reported in Tables 4 and 5. The best result achieved for each dataset is highlighted in bold. The Friedman [69] test is used to compare the results of all methods over the twenty-four classification datasets. The Friedman test is a nonparametric equivalent of repeated-measures ANOVA and is used to compare between several methods across multiple datasets [70]. For each dataset, the Friedman test ranks each algorithm, with the best performing one getting rank 1, the second best rank 2, and so forth. Then, the average rank and its standard deviation are computed considering all the datasets. The best method is the one presenting the lowest average rank. The degrees of freedom (d.f.) for this comparison were set as $d.f. = n - 1$, where n is the total number of methods,

adopting a significance level of $\alpha = 0.05$ for the tests. The Friedman test was performed to verify that there was a significant difference between the approaches. A post hoc Bonferroni–Dunn test was conducted for a pairwise comparison between the ranks achieved by each methods. The post hoc test's findings are depicted using the critical difference diagram (DC) proposed in [71], clarifying the statistical significance between the compared methods. A significant disparity in classifier performance is identified if the average rank difference exceeds the CD, computed as $q_\alpha \sqrt{\frac{k(k+1)}{6N}}$, where the critical value q_α is based on the studentized range statistic divided by $\sqrt{2}$.

The second statistical analysis was conducted in a pairwise fashion to verify whether the difference in classification accuracy obtained by the SGP.DES significantly improved the classification accuracy when compared to the other methods. To that end, the Wilcoxon nonparametric signed-rank test with a level of significance of $\alpha = 0.05$ was used since it was suggested in [70] as a robust method for pairwise comparisons between classification algorithms over several datasets. The results of the Wilcoxon statistical test are shown in the last rows of Tables 4 and 5. Methods that achieved performances equivalent to SGP.DES are marked with '*m*'; those that achieve statistically superior performance are marked with '+', and those with inferior performance are marked with '−'. Additionally, the p values are presented in the last rows of the tables.

To assess the individual contribution of each component of the SGP.DES framework, an ablation study was conducted focusing on the key modules: the prototype selection process, the meta-classifier, and the importance of meta-features. This ablation approach involves systematically removing, identify conditions or modifying each module to understand its impact on the overall performance of the system [72].

The study evaluates the meta-classifier with dynamic selection enabled and disabled and examines the efficiency of prototype selection and its impact on defining competence regions with the KNN algorithm. The Gini index [73], which measures the inequality among values of a frequency distribution—in this case, the distribution of classification errors across different meta-features—is applied across twenty-four datasets. This analysis aims to identify the relevance of each meta-feature in improving the classification accuracy of the SGP.DES framework, especially when using Random Forest as the meta-classifier.

Table 2 Description of the experimental datasets

#	Name	# of Examples	# of Attributes	# of Classes	Repository
1	Appendicitis	106	7	2	KEEL
2	Banana	5300	2	2	KEEL
3	Bupa	345	6	2	KEEL
4	Contraceptive	1473	9	3	KEEL
5	Glass	214	10	7	UCI
6	Haberman	306	3	2	UCI
7	Hayes_Roth	160	4	3	KEEL
8	Heart	270	13	2	UCI
9	Ionosphere	351	33	2	UCI
10	Led7digit	500	7	10	KEEL
11	Marking	6876	13	9	KEEL
12	Monk2	432	6	2	KEEL
13	Movement_libras	360	90	15	KEEL
14	Phoneme	5404	5	2	KEEL
15	Pima	768	8	2	UCI
16	Satimage	6435	36	7	UCI
17	Segment	2310	19	7	KEEL
18	Sonar	208	60	2	UCI
19	Specftheart	267	44	2	KEEL
20	Titanic	2201	3	2	UCI
21	Vowel	900	13	11	KEEL
22	wine	178	13	3	UCI
23	Winequality_Red	1599	11	11	KEEL
24	Winequality_White	4898	11	11	KEEL

Table 3 Parameterization of the methods used in comparative experiments

Method	Parameters and execution mode
SGP.DES	$K_{\text{base}} = 3$, Euclidean distance, $S_\theta = 7$
	$M_\lambda =$ 'I' for KNN, 'R' for RF, 'S' for SVM, $S_C = 50$
	$H_p = [0.1, 0.2, \ldots, 0.9]$, $J_p = [5, 10, \ldots, 35]$
	$D_p = [0.01, 0.001, 0.0001]$, $R_p = [0.25, 0.5, 1, 1.5, 2]$, $\gamma = 0.75$
SVM	Grid search for parameters: regularization C, kernel γ
Random forest	Number of trees: 25 to 200 in steps of 25, best performance on validation
Bagging	Grid search: estimators: [10, 50, 100, 200]
	samples: [0.5, 0.7, 1.0], features: [0.5, 0.7, 1.0]
	bootstrap: [True, False]
Adaboost	Grid search: n_estimators: [10, 50, 100, 200], learning_rate: [0.01, 0.1, 0.5, 1]
DCS	SINGLE-BEST, STACKED, STATIC-SELECTION, LCA, MCB, MLA, OLA, RANK, APRIORI
DES	KNORA-E, KNORA-U, META.DES

Hyperparameters for DCS and DES methods include 50 perceptrons and a neighborhood size of $k = 7$

5 Experimental results

The experiments were conducted to answer to the following research questions: (1) Can the application of multiple criteria using meta-features extracted from representative prototypes in the training data, facilitate the creation of a more effective dynamic selection dataset (DSEL)? Specifically, can it train a meta-classifier capable of employing various learning algorithms, such as those based on instance-based learning or optimized models, to precisely select the best base classifier for a specific test sample? (2) Does the proposed SGP.DES framework surpass the performance of well-established single-model classifiers and static ensemble methods, as well as existing DES methods? This is particularly of interest in scenarios characterized by small datasets that are inadequate for training and optimizing classifier hyperparameters.

5.1 Comparison with static methods

In this section, the experimental results contrast three variants of the SGP.DES framework (SGP.DES.K, SGP.DES.S and SGP.DES.R) with single-model classifiers (the same used as a base in SGP.DES - KNN, RF and SVM parametrized) and five static ensemble methods (Bagging, Adaboost, STACKED, SINGLE-BEST and STATIC-SELECTION). The parameterization of the methods is specified in Table 3, based on a study [74] that ranked the best classifiers and the comparison is conducted on datasets showed in Table 2, both detailed in Subsection 4.

The classification accuracy and the average rank of each method are reported in Table 4. For each dataset, a pairwise comparison was performed between the results obtained by the proposed SGP.DES variants and those obtained by each static ensemble method and single-model classifier using the Wilcoxon test. Moreover, Fig. 2 shows the critical difference diagram, and Fig. 3 demonstrates the variability and consistency of each classifier performance.

5.1.1 Analysis and discussion of static methods results

The results indicated that the SGP.DES framework outperformed both the static ensemble methods and the single model methods in terms of average accuracy and low average rank. Table 4 shows that the SGP.DES.K variant not only achieved the highest average accuracy (82.34%) among all the evaluated methods but also had the lowest average rank (3.02), establishing itself as a robust and reliable classifier in various scenarios. The other SGP.DES variants, SGP.DES.S and SGP.DES.R, also demonstrated solid average performances, with accuracies of 80.14% and 80.26%, respectively.

However, no significant statistical difference was detected according to the Wilcoxon test when the SGP.DES.S and SGP.DES.R variants were compared with those of the SVM and Bagging methods. By examining the results by dataset, the SGP.DES framework stands out in 13 datasets, while the second-ranked method, SVM, excels in 4, followed by 1NN with 2, Random Forest with 2, and Bagging, STACKED and STATIC-SELECTION each with 1. This indicates that different methods may be preferable for different types of classification problems but, in general terms, for the SGP.DES framework demonstrated more consistent results in terms of precision.

This finding is in line with previous literature on dynamic ensemble selection (DES), which has shown that

Table 4 Mean and standard deviation results of the accuracy obtained for the proposed SGP.DES and static classification models

Databases	SGP.DES.K	SGP.DES.S	SGP.DES.R	1NN	SVM	RF	Bagging	Adaboost	STACKED	SINGLE-BEST	STATIC-SELECTION
Appendicitis	88.66(6.31)	87.71(6.44)	87.71(6.44)	83.07(6.98)	**92.92**(6.02)	91.38(6.75)	92.87(7.05)	84.98(6.75)	86.85(3)	92.82(4.09)	91.9(5.67)
Banana	69.05(4.34)	65.5(4.26)	68.1(4.33)	77.61(6.27)	**86.09**(6.6)	61.44(0.18)	83.04(6.24)	65.19(0.18)	58.81(3.13)	61.7(3.36)	59.81(2.59)
Bupa	**90.14**(3.06)	82.32(3.13)	79.42(3.46)	58.26(5.16)	67.92(7.2)	66.32(4.1)	69.78(7.6)	66.52(4.1)	64.2(6.99)	64.82(8.96)	64.2(8.85)
Contraceptive	**80.67**(2.15)	76.64(2.86)	76.98(2.04)	43.45(1.73)	53.1(1.76)	53.84(2.72)	56.37(2.53)	49.49(2.72)	54.77(3.25)	54.48(4.21)	53.39(3.11)
Glass	70.72(9.16)	71.23(9.75)	**75.57**(6.79)	57.6(6.3)	59.9(4.44)	52.64(8.75)	51.66(2.73)	50.06(8.75)	59.3(4.26)	51.85(10.27)	50.08(7.09)
Haberman	**90.45**(4.19)	85.39(4.52)	86.26(5.54)	62.4(3.85)	76.93(2.07)	83.42(1.01)	77.27(3.13)	73.86(1.01)	80.08(4.21)	81.8(3.71)	77.64(1.93)
Hayes-roth	**85.63**(7.18)	79.38(7.19)	81.88(8.09)	59.38(5.85)	69.55(2.57)	58.44(9.53)	69.58(10.46)	61.25(9.53)	59.52(8.67)	62.2(12.62)	56.84(9.11)
Heart	**91.11**(4.42)	87.04(6.42)	86.67(5.14)	76.3(4.79)	86.39(5.34)	89.63(6.98)	83.22(4.34)	82.96(6.98)	89.56(8.22)	85.6(5.77)	86.79(6.06)
Ionosphere	92.04(3.78)	89.47(3.79)	88.06(5.39)	83.47(2.19)	**99.69**(1.56)	92.15(2.83)	93.59(1.82)	87.48(2.83)	93.3(4.08)	89.64(4.02)	87.8(2.58)
Led7digit	75.8(3.7)	72.4(2.3)	77(2.92)	70.8(4.44)	74.47(4.16)	**78.76**(2.92)	74.47(5.22)	76.6(2.92)	78.11(2.83)	75.54(4.22)	75.76(2.59)
Marketing	**44.32**(2.93)	41.45(3.95)	43.93(2.88)	27.78(1.64)	36.02(1.43)	34.56(1.35)	34.01(0.94)	35.38(1.35)	35.43(1.22)	32.9(1.48)	32.98(1.37)
Monk-2	93.51(3.83)	90.97(3.88)	89.35(4.06)	79.16(1.87)	95.83(1.76)	87.08(3.04)	**98.37**(1.95)	81.24(3.04)	83.72(3.47)	84.22(5.11)	85.44(4.67)
Movement_libras	65(3.17)	63.33(3.75)	60.83(3.01)	**76.67**(4.1)	70.44(5.43)	55.92(1.16)	58.76(4.71)	64.17(1.16)	56.47(4.05)	48.74(2.06)	50.83(5.5)
Phoneme	84.43(2.36)	89.5(2.13)	**91.48**(2.62)	79.92(2.7)	86.12(2.01)	82.53(0.67)	83.82(3.59)	77.08(0.67)	80.06(1.67)	81.02(2.11)	80.28(1.47)
Pima	**90.37**(3.93)	84.25(3.17)	88.42(3.94)	67.58(1.85)	80.1(2.91)	80.92(4.56)	79.27(1.49)	79.3(4.56)	79.69(4.02)	78.71(4.5)	79.69(4.78)
Satimage	87.18(4.51)	86.59(3.59)	85.98(2.47)	88.3(0.56)	**94.55**(0.48)	89.18(0.94)	94.25(1.22)	85.41(0.94)	89.76(1.32)	87.05(1.09)	87.15(1.08)
Segment	94.07(1.92)	94.68(2.53)	91.86(2.56)	94.98(0.79)	61.79(1.66)	94.95(1.62)	90.91(2.34)	90.78(1.62)	**95.98**(1.42)	93.24(1.85)	92.97(1.67)
Sonar	94.05(4.2)	87.98(4.86)	87.98(4.09)	87.26(4.1)	92.52(4.35)	**95.96**(4.07)	90.08(3.25)	86.53(4.07)	95.78(5.47)	90.97(5.14)	95.78(4.8)
Spectfheart	**94.63**(3.99)	92.75(3.11)	87.01(3.14)	70.57(6.61)	84.54(0.22)	86.49(0.22)	84.54(0.22)	79.01(0.22)	83.72(1.82)	82.48(2.87)	84.54(0.22)
Titanic	**88.96**(1.85)	85.96(1.3)	88.69(1.58)	78.83(1.39)	83.81(1.78)	85.36(1.71)	83.76(1.76)	77.6(1.71)	83.03(1.71)	83.18(1.52)	83.08(1.67)
Vowel	64.44(7.2)	64.75(6.95)	65.86(4.23)	**89.39**(3.35)	73.06(3.79)	50.66(5.97)	84.09(2.64)	76.67(5.97)	64.53(5.38)	47.01(1.86)	49.93(6.5)
Wine	96.43(2.5)	96.42(3.93)	96.43(2.5)	95.24(3.36)	95.79(4.61)	97.04(2.08)	95.79(4.61)	97.04(2.08)	94.64(5.37)	96.4(3.92)	**97.63**(2.47)
Winequality-red	**75.61**(3.62)	74.23(3.44)	74.67(3.7)	50.84(3.26)	62.23(2.52)	62.67(3.7)	60.62(3.41)	61.59(3.7)	61.43(3.42)	60.83(3.64)	61.36(3.63)
Winequality-white	68.8(1.45)	**73.46**(1.91)	66.11(1.53)	40.67(1.86)	53.59(1.34)	54.87(0.81)	46.38(4.73)	53.28(0.81)	55.01(1.28)	53.39(1.78)	53.43(1.12)
Avg. accuracy	**82.34**(3.99)	80.14(4.13)	80.26(3.85)	70.81(3.54)	76.56(3.58)	74.43(3.24)	76.52(3.67)	72.64(3.24)	74.32(3.76)	72.52(4.17)	72.47(3.77)
Rank	**3.02**	4.45	4.22	8.75	4.91	5.18	5.95	8.22	6.31	7.58	7.35
Wilcoxon-p	n/a	−(0.0025)	−(0.011)	−(0.001)	−(0.0424)	−(0.0005)	−(0.0178)	−(<0.0001)	−(0.0002)	−(<0.0001)	−(<0.0001)
	+(0.0025)	n/a	∼(0.6729)	−(0.0043)	∼(0.1875)	−(0.0095)	∼(0.0839)	−(0.0002)	−(0.0031)	−(0.0002)	−(0.0003)
	+(0.011)	∼(0.6729)	n/a	−(0.0025)	∼(0.1433)	−(0.0105)	∼(0.0893)	−(0.0001)	−(0.002)	−(0.0002)	−(0.0003)

The best results are in bold

Table 5 Mean and standard deviation results of the accuracy obtained for the proposed SGP-DES and 9 state-of-the-art dynamic selection techniques

Dataset	SGP.DES.K	SGP.DES.S	SGP.DES.R	KNORA-E	KNORA-U	META.DES	LCA	MCB	MLA	OLA	DCS-APRIORI	DCS-RANK
Appendicitis	88.66 (6.31)	87.71 (6.44)	87.71 (6.44)	92.82 (7.12)	91.9 (5.67)	**93.84** (5.49)	88.89 (6.99)	92.82 (6.28)	88.89 (6.99)	91.81 (4.88)	90.88 (6.09)	91.81 (5.92)
Banana	**69.05** (4.34)	65.5 (4.26)	68.1 (4.33)	65.65 (4.15)	65.67 (4.08)	65.71 (4.07)	64.79 (3.7)	65.83 (4.22)	64.79 (3.7)	65.67 (4.04)	59.34 (1.56)	65.3 (3.84)
Bupa	**90.14** (3.06)	82.32 (3.13)	79.42 (3.46)	62.34 (5.17)	63.89 (9.36)	65.1 (6.77)	61.72 (7.9)	62.96 (6.53)	61.72 (7.9)	62.34 (8.78)	62.34 (5.75)	61.72 (5.46)
Contraceptive	**80.67** (2.15)	76.64 (2.86)	76.98 (2.04)	51.94 (1.72)	53.02 (2.69)	53.53 (3.57)	50.48 (1.44)	52.52 (1.61)	50.7 (1.47)	53.54 (3.11)	51.36 (1.31)	52.37 (1.57)
Glass	70.72 (9.16)	71.23 (9.75)	**75.57** (6.79)	57.53 (10.5)	53.52 (2.72)	53.52 (6.05)	50.05 (4.04)	59.83 (9.29)	50.05 (4.04)	54.61 (8.73)	49.49 (7.1)	55.8 (8.85)
Haberman	**90.45** (4.19)	85.39 (4.52)	86.26 (5.54)	80.42 (3.61)	79.39 (4.09)	79.02 (4.5)	77.63 (3.15)	80.41 (5.17)	77.63 (3.15)	80.41 (5.19)	79.04 (2.15)	79.72 (4.18)
Hayes-roth	**85.63** (7.18)	79.38 (7.19)	81.88 (8.09)	78.24 (7.2)	76.84 (13.44)	74.9 (6.09)	47.48 (10.22)	62.19 (6.48)	47.48 (8.95)	63.53 (5.85)	57.51 (6.77)	77.58 (6.01)
Heart	**91.11** (4.42)	87.04 (6.42)	86.67 (5.14)	84.41 (3.84)	86 (5.34)	85.99 (5.65)	83.62 (6.6)	84.41 (5)	83.62 (6.6)	85.6 (5.46)	86 (5)	84.81 (5.77)
Ionosphere	**92.04** (3.78)	89.47 (3.79)	88.06 (5.39)	89.33 (2.88)	89.03 (3.31)	89.64 (3.05)	84.75 (4.08)	89.94 (4.41)	84.75 (4.08)	89.33 (2.5)	89.32 (4.08)	89.33 (3.22)
Led7digit	75.8 (3.7)	72.4 (2.3)	77 (2.92)	**78.75** (3.29)	78.04 (3.61)	75.76 (5.31)	76.61 (3.85)	77.47 (3.85)	76.61 (3.85)	77.04 (3.39)	75.54 (3.29)	77.25 (2.86)
Marketing	**44.32** (2.93)	41.45 (3.95)	43.93 (2.88)	33.3 (1.32)	33.87 (1.49)	33.05 (1.24)	33.36 (1.64)	33.05 (1.26)	33.37 (1.65)	33.33 (1.58)	32.51 (0.9)	32.95 (0.94)
Monk-2	**93.51** (3.83)	90.97 (3.88)	89.35 (4.06)	89.41 (3.8)	88.94 (4.14)	91.14 (6.7)	85.46 (4.25)	88.17 (5.05)	85.46 (4.25)	89.17 (2.69)	85.45 (4.26)	89.16 (1.97)
Movement_libras	65 (3.17)	63.33 (3.75)	60.83 (3.01)	66.28 (4.46)	66.8 (3.75)	**66.87** (5.01)	52.61 (5.78)	61.82 (3.34)	52.61 (5.78)	58.55 (3.34)	49.34 (3.46)	64.5 (4.46)
Phoneme	84.43 (2.36)	89.5 (2.13)	**91.48** (2.62)	81.28 (1.46)	80.92 (1.62)	87.16 (1.75)	81.16 (1.21)	81.16 (1.15)	81.16 (1.21)	81.5 (1.33)	80.94 (1.65)	80.96 (1.28)
Pima	**90.37** (3.93)	84.25 (3.17)	88.42 (3.94)	80.52 (3.7)	79.69 (3.97)	80.1 (3.32)	77.05 (3.74)	79.97 (2.69)	77.05 (3.74)	79.69 (2.92)	79.41 (2.95)	80.11 (1.66)
Satimage	87.18 (4.51)	86.59 (3.59)	85.98 (2.47)	87.81 (0.95)	87.21 (0.99)	87.35 (0.99)	87.44 (0.83)	87.65 (1.11)	87.44 (0.83)	**87.93** (1.23)	86.89 (1.06)	87.64 (1.21)
Segment	94.07 (1.92)	94.68 (2.53)	91.86 (2.56)	94.63 (1.89)	93.94 (1.89)	94.36 (1.85)	94.31 (1.93)	**94.96** (2.35)	94.45 (1.87)	**94.96** (2.1)	93.61 (1.92)	94.31 (1.94)
Sonar	94.05 (4.2)	87.98 (4.86)	87.98 (4.09)	97.4(5.66)	96.99 (4.48)	**98.19** (6.14)	86.73 (13.21)	94.93 (6.04)	86.73 (13.21)	95.75 (5.48)	94.17 (6.02)	96.61 (4.13)
Spectfheart	**94.63** (3.99)	92.75 (3.11)	87.01 (3.14)	83.31 (2.22)	84.94 (1.68)	84.53 (1.5)	84.54 (0.22)	80.43 (6.88)	84.54 (0.22)	81.66 (3.34)	83.31 (1.76)	80.85 (2.26)
Titanic	**88.96** (1.85)	85.96 (1.3)			83.33 (1.86)	83.91 (1.64)	82.6 (1.85)		82.6 (1.85)			

Table 5 (continued)

Dataset	SGP.DES.K	SGP.DES.S	SGP.DES.R	KNORA-E	KNORA-U	META.DES	LCA	MCB	MLA	OLA	DCS-APRIORI	DCS-RANK
			88.69 (1.58)	83.57 (1.93)				83.91 (1.43)		83.91 (1.43)	83.57 (1.98)	83.57 (1.93)
Vowel	64.44 (7.2)	64.75 (6.95)	65.86 (4.23)	**67.77** (4.58)	66.55 (4.69)	67.2 (4.93)	56.2 (2.83)	63.77 (3.9)	56.1 (2.63)	62.26 (5.23)	51.12 (4.72)	67.44 (3.77)
Wine	96.43 (2.5)	96.42 (3.93)	96.43 (2.5)	96.44 (3.25)	**97.04** (2.94)	**97.04** (2.94)	95.79 (3.47)	96.42 (2.5)	95.79 (3.47)	94.62 (4.94)	97.02 (2.99)	94.62 (4.94)
Winequality-red	**75.61** (3.62)	74.23 (3.44)	74.67 (3.7)	61.23 (3.75)	60.76 (3.37)	71.83 (3.81)	60.02 (3.39)	61.03 (3.21)	59.95 (3.4)	60.89 (3.28)	59.82 (2.65)	60.29 (2.94)
Winequality-white	68.8 (1.45)	**73.46** (1.91)	66.11 (1.53)	52.76 (0.79)	53.59 (1.29)	70.61 (0.69)	52.67 (1.75)	53.11 (0.99)	52.67 (1.74)	52.56 (1.75)	53.13 (1.65)	52.08 (1.16)
Avg. accuracy	**82.34** (3.99)	80.14 (4.13)	80.26 (3.85)	75.71 (3.72)	75.49 (3.85)	77.1 (3.88)	71.5 (4.09)	74.53 (3.95)	71.51 (4.02)	74.19 (3.86)	72.13 (3.38)	75.03 (3.43)
Rank	**3.52**	4.72	4.81	5.14	6.10	4.85	9.7	6.16	9.6	6.58	9.41	7.35
Wilcoxon-p	n/a	−(0.0025)	−(0.011)	−(0.0017)	−(0.0004)	−(0.0178)	−(<0.0001)	−(<0.0001)	−(<0.0001)	−(<0.0001)	(<0.0001)	−(0.0002)
	+(0.0025)	n/a	∼(0.6729)	−(0.0424)	−(0.0229)	∼(0.1073)	−(<0.0001)	−(0.0074)	−(<0.0001)	−(0.002)	−(0.0003)	−(0.0126)
	+(0.011)	∼(0.6729)	n/a	−(0.0269)	−(0.0314)	∼(0.1355)	−(<0.0001)	−(0.0043)	−(<0.0001)	−(0.002)	−(0.0308)	−(0.0087)

The best results are in bold

dynamic selection typically surpasses static combination rules in many applications [51]. This advantage is particularly notable when a pool of weak linear classifiers is used, as they specialize in different regions of the feature space. As noted in [75], a static combination of base classifiers may not yield optimal performance, as consensus on the correct answer among the classifiers in the pool may be lacking. In contrast, dynamic selection involves only the most competent classifiers for a given test sample, preventing less relevant classifiers from negatively influencing the ensemble decision. The SGP.DES.K variant achieved the lowest average rank when compared to the other methods. Although, in the critical difference diagram in Fig. 2, the performance of the SGP.DES.K is statistically equivalent to the SVM classifier, and the SGP.DES.S and the SGP.DES.R variants are statistically comparable to the SVM classifier, Bagging, and Random Forest methods, based on both the Friedman test and Bonferroni–Dunn post hoc test. However, Table 4 revealed that the Wilcoxon signed-rank test, at a significance level of $\alpha = 0.05$, indicated that the SGP.DES.K variant outperforms all other methods, including SVM and other SGP.DES variants, which are equivalent to SVM and Bagging methods. Consequently, the analysis demonstrated that the classification performance of the proposed SGP.DES framework on the twenty-four well-known datasets was competitive with the best traditional algorithms recognized in the literature.

The analysis of precision variability revealed that methods such as SVM can reach high performance peaks, such as a maximum accuracy of 99.69% on the Ionosphere dataset, but they exhibit greater variability in results than does the SGP.DES framework. These findings suggested that the performance of SGP.DES across different classification problems is more consistent. Similarly, methods such as 1NN, Random Forest, STATIC-SELECTION, SINGLE-BEST, and STACKED Classifier exhibit significant variability, as evidenced by higher standard deviations and wider interquartile ranges in the box plot visualizations in Fig 3.

Notably, despite there not being significant statistical differences in overall performance among the SGP.DES variants, the precision of these variants varied across specific datasets, including Phoneme, Winequality-white, Led7digit, Vowel, and Glass. Among these, SGP.DES.R and SGP.DES.S consistently outperformed SGP.DES.K. This indicates potential benefits in selecting specific variants for certain classification challenges.

The observed favorable results of SGP.DES compared to single models and static ensembles in this comparative study can largely be attributed to the nature of the datasets used, which are predominantly small and involve more complex classification problems. In such cases, the

available training data may not provide enough samples to effectively train a single classifier model or to accurately select the best hyperparameters or adapt efficiently to correctly classify instances located in regions of indecision. Here, the SGP.DES framework offers two distinct advantages: (1) its pool comprises linear classifiers that do not require hyperparameter selection, facilitating efficient training on small datasets. Due to the relatively small size of the training sets, the classifiers can become specialized in specific local regions of the feature space. Through dynamic selection, only the classifiers deemed most competent in the local region of a specific test sample are employed for its label prediction. And (2) the use of prototypes that select more cohesive regions of the dataset, excluding noisy data in certain borderline areas. This advantage of using instance selection techniques is strictly related to the benefits these techniques provide where the KNN algorithm is used to define the competence regions in dynamic selection methods, as reported in previous studies [29, 30]. Consequently, dynamic ensemble selections (DESs) enable the achievement of high classification accuracy, even in scenarios where the problems are poorly defined.

5.2 Comparison with state-of-the-art DES methods

In this section, the proposed SGP.DES Framework is contrasted with nine state-of-the-art dynamic selection methods. The goal of this analysis is to determine whether the performance of the proposed system is significantly superior to that of state-of-the-art DES methods.

A comparative analysis of the experimental results obtained by the three variants of the SGP.DES framework: SGP.DES.K, SGP.DES.R, and SGP.DES.S. These methods are compared with six other dynamic classifier selection (DCS) methods—LCA, MCB, MLA, OLA, DCS-RANK, and DCS-APRIORI, and three dynamic ensemble selection (DES) methods—KNORA-E, KNORA-U and META.DES. The methods were applied to well-known public datasets, as shown in Table 2. These methods were selected for their outstanding results in the dynamic selection literature [6].

The same pool of classifiers (M = 50) is used for all methods to ensure a fair comparison. For all methods, the size of the region of competence, K, was set at 7, as this value achieved the best result in previous studies comparing different DES methods [22, 46].

5.2.1 Analysis and discussion with state-of-the-art DES methods

The results are shown in Table 4. For each dataset, a pairwise comparison was performed between the results obtained by the proposed SGP.DES and those obtained by each state-of-the-art DS method. Additionally, the average rank of each method, as well as the results of the sign test, is presented at the end of Table 5. Figure 4 illustrates the average rank of each method using the critical difference (CD) diagram. Like in Sect. 5.1, the CD was calculated using the Bonferroni–Dunn post hoc test. Figure 5 demonstrates the variability and consistency of the performance of each DS method's performance.

The SGP.DES variants, particularly SGP.DES.K, demonstrated superior performance with the highest average accuracy and the lowest average rank. SGP.DES.S and SGP.DES.R also achieved good average performance in terms of accuracy and rank. Furthermore, the analysis of the Wilcoxon signed-rank test, presented in the last line of Table 5, indicates that SGP.DES.K significantly outperformed the other methods, considering an $\alpha = 0.05$. However, the variants SGP.DES.S and SGP.DES.R are

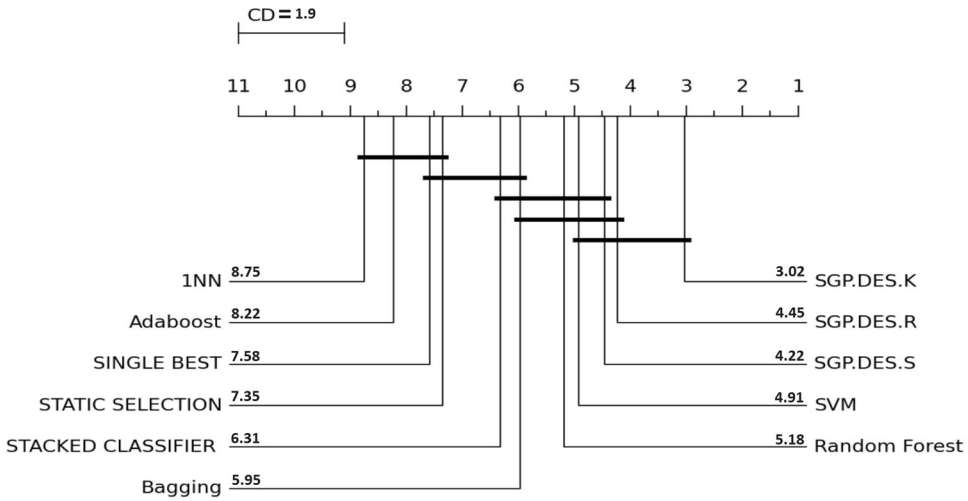

Fig. 2 Average rank of the static methods and SGP.DES variants across the twenty-four datasets. The best method is identified by the lowest average rank

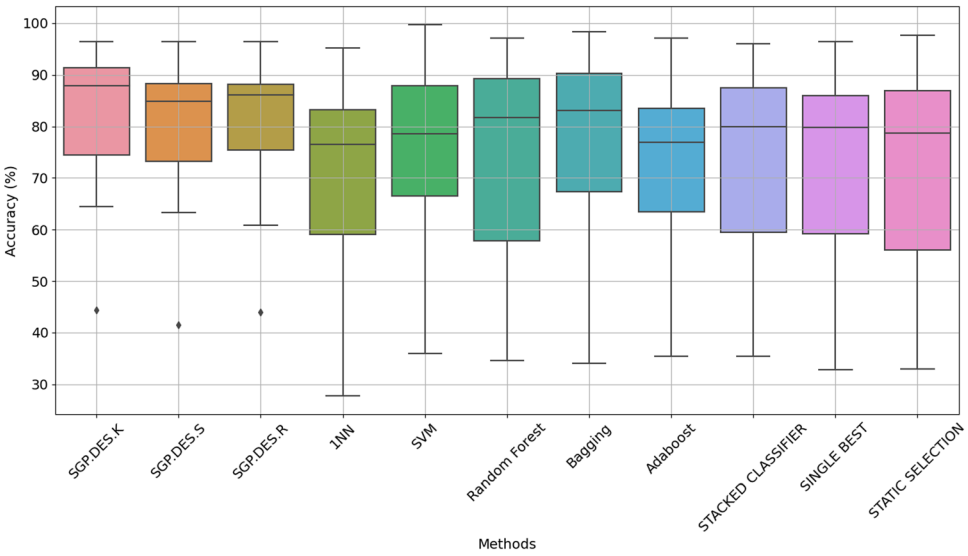

Fig. 3 Accuracy distribution for each classification method. The boxes represent the interquartile range of accuracy, the lines within the boxes denote the median, and the points represent outlier values. This visualization underscores the variability and consistency of each classifier's performance

statistically equivalent to the META.DES Framework. In the critical difference diagram in Fig. 4, despite the great distance from other methods and variants, the performance of SGP.DES.K is statistically equivalent to the variant SGP.DES.S and SGP.DES.R, META.DES and KNORA-E are connected by the black line. The SGP.DES.S and SGP.DES.R variants are statistically comparable to those of the META.DES and KNORA-E are short distances but relatively great distances from the KNORA-U, OLA, and MCB methods.

In terms of variability, Fig. 9, as measured by the standard deviation, the SGP.DES variants exhibited relatively high consistency. This suggests that, in addition to having the highest average accuracy, the SGP.DES variants are also consistent in their performance across different datasets compared to other DCS and DES methods.

When performing pairwise comparisons between the methods, the SGP.DES framework outperformed the other DCS and DES methods. Its performance is statistically superior to that of any of the other nine state-of-the-art methods. This can be explained by two factors: most state-of-the-art DCS and DES methods rely on a single criterion to estimate the competence of the base classifier, such as local accuracy, ranking, and classifier behavior. Additionally, even though SGP.DES does not incorporate a feature selection implementation akin to META.DES.Oracle [22] through the BSPO algorithm, uses of multiple meta-features, as shown in Table 1, along with various operational possibilities for the meta-classifier using different embedded classification algorithms, enable SGP.DES to effectively select the most promising base classifier for a specific test sample, in contrast to other DS methods.

The analysis of the experimental results indicated that the variant SGP.DES.K, which uses the instance-based mode in operating the meta-classifier of SGP.DES on meta-features data, proved to be more promising for the dynamic selection of the best methods, using prototypes generated by the A-SGPE algorithms. However, it is important to note that the performance of SGP.DES.K may vary according to the characteristics of the datasets, and in some cases, other DCS or DES methods, especially the META.DES or other variants of the SGP.DES, demonstrate equivalent or superior performance.

These findings indicate that integrating the prototype selection method A-SGPE with a configurable meta-classifier, which leverages the choice of different predictive algorithms to select the most effective base classifiers for a given instance, substantially enhances the adaptability and precision of the dynamic selection process. To our knowledge, at the time of this study, this specific approach is not documented in the existing literature on dynamic selection methods. This enhancement is particularly significant during the generalization phase, distinguishing it from other DES methods assessed in this research. However, the framework exhibits limitations that merit further exploration. Future research could involve incorporating the framework with diverse prototype selection techniques from the literature to optimize the DSEL set more effectively for specific classification challenges. Additionally, implementing a feature selection strategy to identify the most important meta-features for a given classification scenario, akin to the Meta-DES.Oracle approach [22], would be advantageous. Furthermore, developing an automatic selector mode for the meta-classifier that determines the optimal predictive algorithm during the meta-

training phase could streamline the selection process, thus eliminating the manual selection of different variants of SGP-DES via the hyperparameter M_λ. These proposed enhancements are expected to improve adaptability to the problem at hand, facilitate optimization, and enhance the overall performance of the framework for various classification scenarios.

5.3 Framework steps ablation of SGP.DES

To assess the individual contribution of each component of the SGP-DES framework, an ablation study was conducted focusing on the key modules: prototype selection process, meta-classifier, and the importance of meta-features. The ablation approach involves systematically removing or modifying each module to understand its impact on the overall performance of the system [72].

5.3.1 Ablation of the prototype selection step

The prototype selection step is crucial for the performance of SGP-DES, as it creates the DSEL set used by the dynamic selection within the SGP-DES framework. It aims to eliminate instances highly likely to be noise, reduce class overlap in regions of confusion, and maintain the statistical properties of the original training set. In Fig. 6, some plots show a granular analysis of the impact of prototype selection on the Compound benchmark dataset [76]. The top-left plot reveals the 'Original Dataset' in terms of its raw complexity, with the data points' diverse shapes and hues representing different classes. Side by side, the 'Prototypes' plot simplifies this complexity, distilling the dataset to its essence with fewer data points that serve as class exemplars.

The bottom plots present a dual perspective: '1NN - prototype decision boundary and original dataset' and '1NN - prototype decision boundary'. These comparisons underscore the results of employing a 1-Nearest Neighbor algorithm that uses the generated prototypes to define the decision boundaries. The Voronoi tessellation, prominently featured in these plots, segments the plane into distinct regions tied to each prototype. This methodological approach guarantees that any novel instance is attributed to the nearest prototype, thereby maintaining the fidelity of the original dataset's structure even after considerable data reduction.

In addition, these visual aids validate the robustness of the data reduction strategy. The Voronoi diagrams, in particular, facilitate a comparative analysis between the original and the reduced datasets, demonstrating the latter ability to preserve the overall distribution and the decision boundaries of the former. Notably, in Fig. 7, the red circles that pinpoint the prototypes encapsulate dense clusters of instances from identical classes, affirming that the reduced dataset provides a well-balanced and precise foundation for subsequent model construction. This visual evidence underscores the potential of the proposed method for achieving significant data reduction without sacrificing the integrity and representativeness of the dataset.

Carrying out analyses on the impact of the prototype, taking as a reference the KNN, without the prototype action and the SGP.DES.K which has the prototype action. Analyzing the results of Table 6 and creating ranges of accuracy such as being less than 70%, between 70% and 90% and greater than 90%, it allows verifying that the performance of the proposal in situations where KNN has more difficulty in classifying, probably in regions with greater uncertainty in the decision boundary, the proposed approach performs better, as illustrated in Table 6. On the

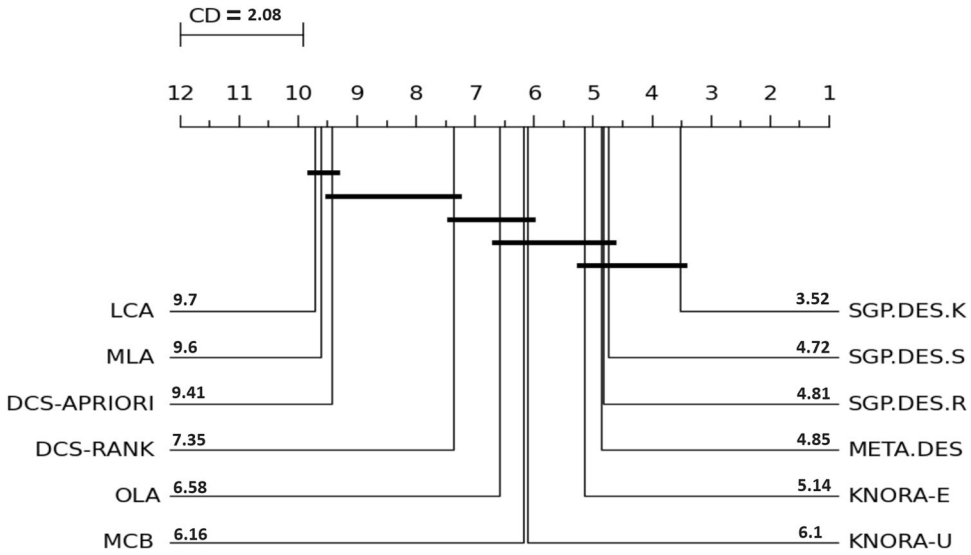

Fig. 4 Average rank of the dynamic selection methods across the twenty-four datasets. The best method is the one with the lowest average rank

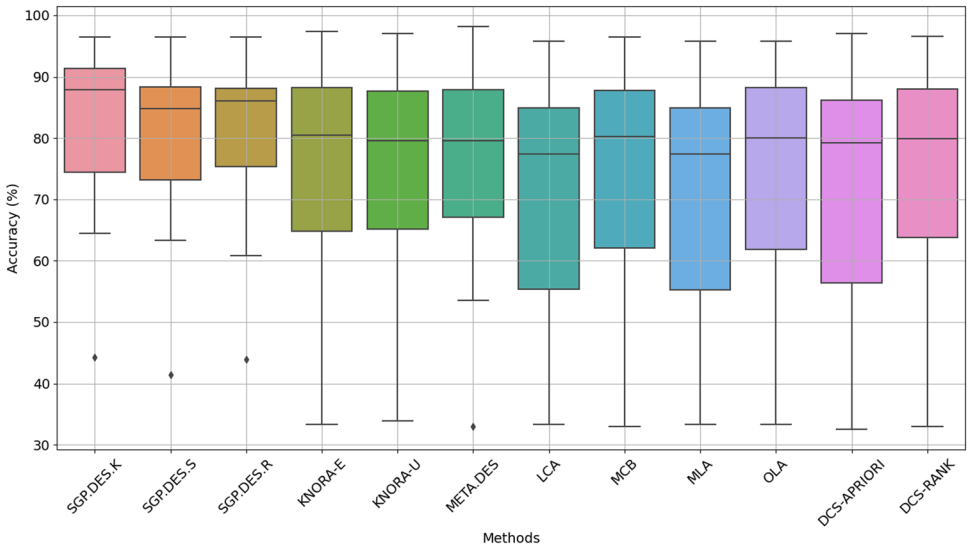

Fig. 5 Corrected precision distribution for each DES method. The boxes represent the interquartile range of accuracy, and the lines within the boxes indicate the median. The SGP.DES methods maintain a leading position not only in terms of average but also in consistency, as indicated by the narrower interquartile ranges

other hand, situations in which KNN has accuracy performance above 90% the SGP.DES is not effective. Finally, in accuracy performance between 70% and 90%, the framework has the majority of the best accuracy performance (9 datasets), but there are some cases in which KNN is better than SGP.DES (4 datasets).

5.3.2 Meta-classifier

The meta-classifier integrates information from meta-features extracted from the prototypes to select the best base classifiers. To evaluate its impact, I compared the default SGP-DES configuration with a version where the meta-classifier was replaced by a simple voting among the classifiers.

Figure 8 shows that the dynamic selection of the meta-classifier improves accuracy compared to the simple majority voting of the base classifiers 'Dynamic selection Disabled'. The accuracy is consistently higher with dynamic selection across all tested datasets, indicating an improvement in model performance. The variability in accuracy measurements also tends to be lower with dynamic selection, suggesting more reliable results. The experiment was conducted with cross-validation, and the bars indicate the standard deviation of the results.

The overall average accuracy of SGP-DES with DS enabled was 82.34%, while that of SGP-DES with DS disabled was 60.65%, an improvement of 21.69 percentage points in favor of SGP-DES with DS enabled. The standard deviation of SGP-DES accuracies with DS enabled was 12.99, compared to 21.55 for SGP-DES with DS disabled, indicating that SGP-DES with DS enabled is more consistent. SGP-DES with DS enabled outperformed DS disabled in all 24 tested datasets, highlighting its superiority.

The graphs reinforce these observations: The boxplot of accuracies shows that SGP-DES with DS enabled is consistently more accurate and less variable. The barplot of average accuracies per dataset with error bars confirms that SGP-DES with DS enabled outperforms DS disabled in all analyzed cases.

It was observed that SGP-DES with DS enabled presents a significant advantage in datasets with a larger number of classes. The correlation between the accuracy difference and the number of classes was 0.79, indicating a positive trend. Notable examples include Movement_libras and Vowel, with 15 and 11 classes, respectively, where SGP-DES with DS enabled presented performance differences of 52.50 and 45.85 percentage points. On the other hand, the correlation between accuracy difference and the number of instances was weak (−0.18), suggesting that the number of instances does not significantly influence the performance difference between the methods.

Despite the advantages, SGP-DES with DS enabled did not show such a significant improvement in datasets with a smaller number of classes, such as Appendicitis and Banana. Furthermore, some datasets, like Glass and Hayes-roth, showed a relatively high standard deviation for SGP-DES with DS enabled, indicating greater variability in the results.

These results demonstrate that the dynamic selection of the meta-classifier in SGP-DES not only improves the overall accuracy of the model but also contributes to more consistent results. This is especially relevant in more complex problems with a larger number of classes, highlighting the effectiveness of this method compared to simple voting among base classifiers.

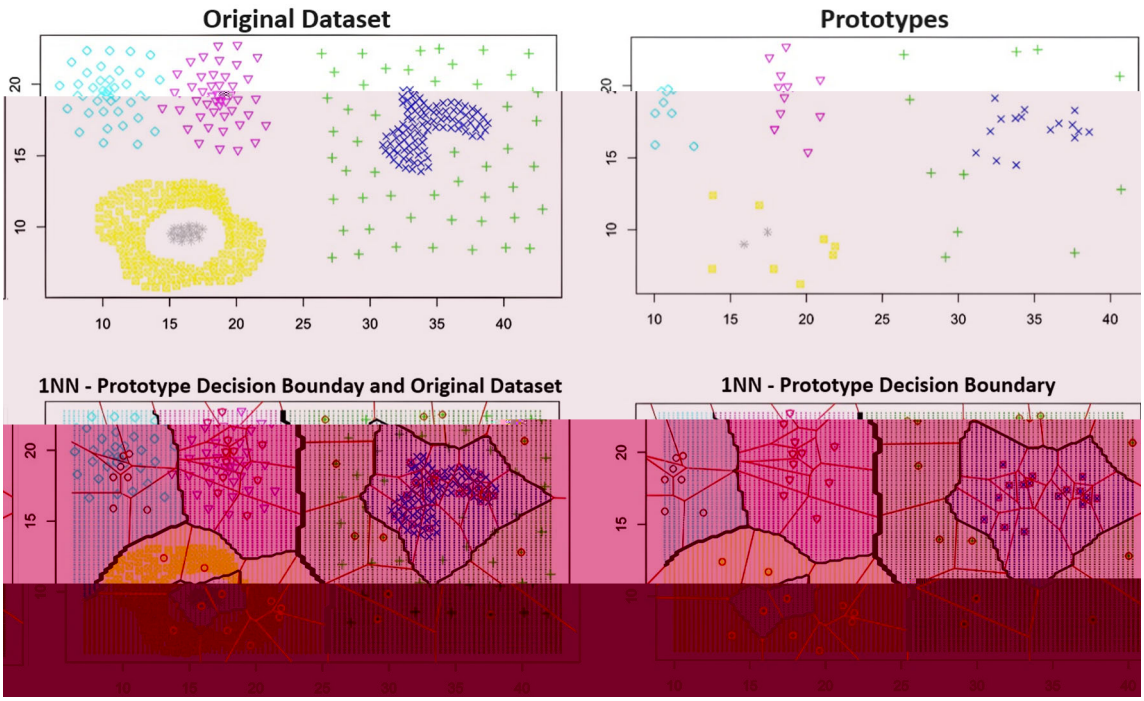

Fig. 6 Instance selection process and 1NN classification—compound dataset

5.3.3 Analysis of meta-feature Importance

To assess the relative importance of meta-features in classifier efficacy, we employed the Gini Index as a measure of impurity to quantify the individual contribution of each meta-feature across the analyzed datasets. Figure 9 presents a heatmap that illustrates the Gini indices for seven distinct meta-features across twenty-four datasets.

Through the heatmap, we observed that the meta-feature `f_cond` exhibited elevated Gini Index values across various datasets, indicating significant importance in class distinction. This result suggested that `f_cond` is a robust and influential meta-feature that consistently contributes to the performance of classification models.

Furthermore, we found that certain datasets such as Banana and Wine demonstrated a more uniform distribution of importance among meta-features, reflecting the need for a balanced combination of information for effective classification. In contrast, datasets such as Appendicitis were dominated by one or two meta-features, which may indicate intrinsic data characteristics.

The chart also revealed specific datasets where individual meta-features, such as `f_cond_support` in Heart and `f_dist_instance` in Spectfheart, had substantially high Gini Index values. The results suggested that these meta-features play a critical role and can be decisive in modeling these particular datasets.

These findings highlight the relevance of a detailed analysis of meta-features in classifier selection. Moreover,

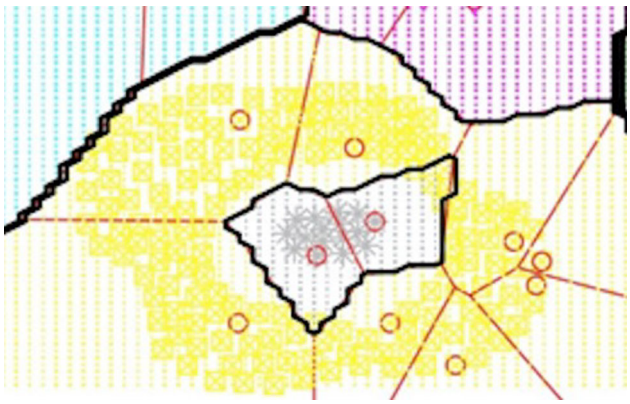

Fig. 7 Zoom out—compound dataset

they demonstrated that the importance of meta-features can vary between datasets. Different problems are associated with varying degrees of data complexity [43, 77] and may require a distinct set of meta-features to obtain a meta-classifier that exhibits behavior closer to that of the Oracle for estimating the competence of the base classifiers.

Table 6 Acc means accuracy results summarized from Table 4

	$Acc < 70\%$	$70\% \leq Acc < 90\%$	$Acc \geq 90\%$
SGP.DES.K	9	9	1
KNN	0	4	1

Hence, the results corroborate the idea that the choice of the best set of meta-features are problem-dependent, as observed in the article on the Meta-DES algorithm [51].

6 Conclusion and future work

In this paper, we introduce a novel dynamic ensemble selection (DES) framework, the self-generating prototype ensemble selection (SGP.DES), which optimizes the dynamic selection dataset (DSEL) through prototype selection and uses a meta-learning approach to dynamically select the most promising base classifiers for accurately predicting the class of a test sample. We proposed seven sets of meta-features, including local accuracy, probabilistic measures, and confidence, sourced from DES literature, to dynamically estimate the competence of base classifiers. The meta-classifier in SGP.DES can operate across multiple learning paradigms, incorporating model-based methods such as Random Forest and SVM, as well as instance-based method with KNN.

Our experiments on twenty-four classification problems compare our framework against static ensembles, single-model classifiers, and current DCS and DES methods. The results showed that the SGP.DES.K variant, based on KNN for dynamic selection, outperformed the others with 82.34% accuracy and the lowest average rank. The SGP.DES.R and SGPDES.S variants had accuracies of 80.26% and 80.14%, respectively. Statistical tests confirmed that the SGP.DES.K had superior performance, demonstrating its effectiveness for dynamic ensemble selection across various scenarios.

However, the SGP.DES.R and SGP.DES.S variants demonstrated the lowest variability in performance. For certain classification challenges, such as those encountered with the Winequality-white and Phoneme datasets, they exceeded the performance of the SGP.DES.K variant. This emphasizes the necessity of selecting the optimal meta-classifier operation mode that aligns with the distinct characteristics of each problem.

The ablation study revealed that each component of the SGP-DES framework significantly improves performance: Prototype Selection enhanced accuracy in challenging regions, with SGP.DES.K outperforming KNN in lower accuracy ranges (10 datasets below 70% and 9 between 70–90%); the meta-classifier increased overall accuracy by 21.69 percentage points (average 82.34% vs. 60.65%) and reduced the standard deviation from 21.55 to 12.99; Meta-features, such as f_{cond}, proved crucial, particularly in complex datasets, with a correlation of 0.79 between accuracy difference and number of classes. These results indicate that prototype selection, the meta-classifier, and selected meta-features are all essential for the effectiveness of SGP-DES. Their removal or simplification leads to noticeable performance deterioration, emphasizing the necessity of each component within the proposed system.

The analysis of meta-feature importance, particularly the Conditional Competence Region Score (f_{cond}), was highly relevant across the datasets considered. However, in certain datasets, such as the Wine, Sonar, Led7digit, Ionosphere, Monk-2, Movement-libras, Spectfheart, and

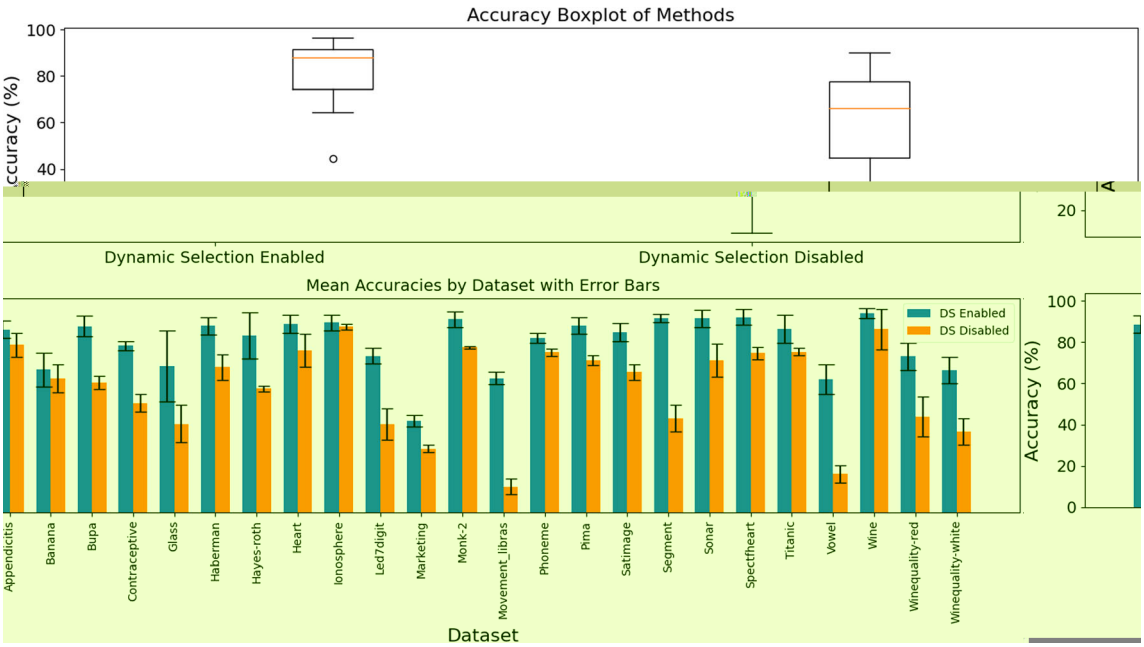

Fig. 8 Comparing the performance of SGP-DES with dynamic selection enabled and disabled on public datasets

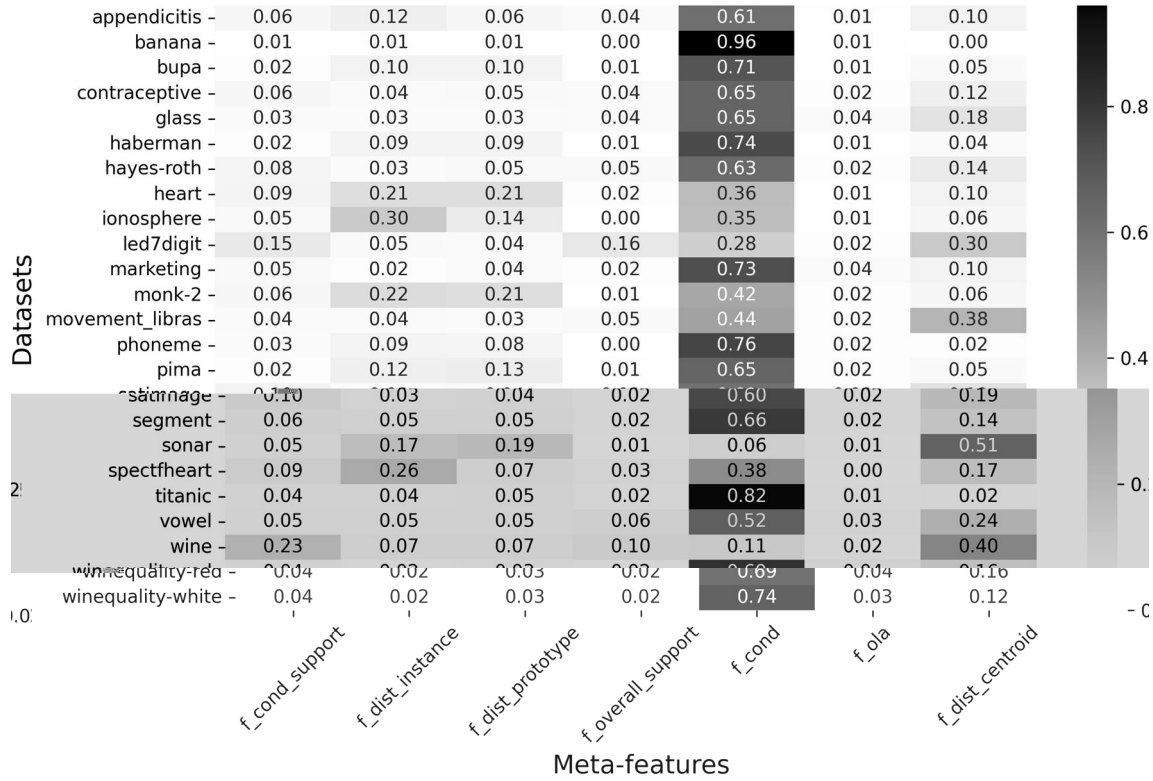

Fig. 9 Heatmap of Gini Index values for meta-features across datasets

Vowel datasets, the significance of a range of meta-features for the meta-classifier was more evenly distributed. This emphasizes the necessity for a thorough meta-feature analysis in classifier selection, illustrating that the relevance of meta-features varies among different datasets. Moreover, these findings highlight the importance of employing adaptive strategies in the construction of the meta-classifier, such as applying feature selection to identify the most discriminant meta-features for a given classification problem. The complexity of various problems requires the use of distinct meta-features for optimal meta-classifier performance, supporting the contention that meta-feature selection should be problem-specific.

In conclusion, the results of the present study confirm SGP.DES is effective in dynamic selection, demonstrating their superior performance over other DCS, DES, and static ensemble methods, such as Random Forest, in terms of average accuracy and ranking. Furthermore, the SGP.DES has proven its ability to tackle a wide range of classification issues across diverse domains. This approach delivers results that rival those of the best-performing classifiers, surpassing both static ensembles and single-model methods, particularly in scenarios involving ill-defined problems or small datasets. This underscores the advantage of DES methods in these contexts.

Future work will focus on the following steps: (1) Incorporate new sets of meta-features into the proposed framework based on the literature [21–23] to better estimate the competence levels of base classifiers and to simplify the optimization process of the meta-classifier among different operational modes, automating the selection process between model-based and instance-based approaches. This includes integrating a broader variety of classification algorithms within the SGP.DES meta-classifier. (2) Implement a feature selection approach during the training phase to select only the meta-features that are relevant to optimize the meta-classifier for a given classification problem. (3) Expand the SGP.DES evaluation to classification problems within ill-defined domains, such as medical datasets. (4) Apply the framework with other paradigms of instance selection techniques [29] to verify if different prototype selection approaches can enhance the accuracy of defining the competence region and consequently improve the quality of the meta-classifier decision on the most competent base classifiers for different problem domains.

By pursuing these directions, the objective is to refine the capabilities of SGP.DES and solidify its role as a leading method in the dynamic selection of ensemble classifiers. These efforts are expected to push the boundaries of what is currently achievable in ensemble learning, addressing the pressing needs of real-world applications that require highly accurate and adaptable classification solutions.

Finally, it is important to analyze the applicability of SGP.DES in more complex and poorly defined domains. A specific hypothesis to be tested is the framework performance in generalization, which is a crucial feature for applications susceptible to drift models.

Acknowledgements We gratefully acknowledge the editorial committee for their support and the anonymous reviewers for their insightful comments and suggestions that have notably improved this manuscript.

Author contributions All the authors have contributed equally to this work. They have collaboratively participated in the development of the study's conception and design, the analysis and interpretation of data, the drafting of the manuscript, and the critical revision for important intellectual content. All the authors have read and approved the final version of the manuscript.

Funding This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior—Brasil (CAPES)—Finance Code 001 and MackPesquisa—Research Support from the Universidade Presbiteriana Mackenzie.

Data availability statement All discussed data are available online at: KEEL: https://sci2s.ugr.es/keel/category.php?cat=clas Fränti, P., Sieranoja, S.: K-means properties on six clustering benchmark datasets (2018): http://cs.uef.fi/sipu/datasets/ UCI repository: https://archive.ics.uci.edu/ml/index.php.

Code availability The code related to this work is available at: https://github.com/manastarla/sgpdes.git.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

1. Dong X, Yu Z, Cao W, Shi Y, Ma Q (2020) A survey on ensemble learning. Front Comp Sci 14:241–258
2. Sesmero MP, Iglesias JA, Magán E, Ledezma A, Sanchis A (2021) Impact of the learners diversity and combination method on the generation of heterogeneous classifier ensembles. Appl Soft Comput 111:107689
3. Kuncheva LI (2014) Combining pattern classifiers: methods and algorithms, 2nd edn. Wiley, Hoboken
4. Suárez-Cetrulo AL, Quintana D, Cervantes A (2023) A survey on machine learning for recurring concept drifting data streams. Expert Syst Appl 213:118934
5. Zyblewski P, Sabourin R, Woźniak M (2021) Preprocessed dynamic classifier ensemble selection for highly imbalanced drifted data streams. Inf Fusion 66:138–154
6. Cruz RM, Sabourin R, Cavalcanti GD (2018) Dynamic classifier selection: recent advances and perspectives. Inf Fusion 41:195–216
7. Britto AS Jr, Sabourin R, Oliveira LE (2014) Dynamic selection of classifiers-a comprehensive review. Pattern Recognit 47(11):3665–3680
8. Breiman L (1996) Bagging predictors. Mach Learn 24(2):123–140
9. Schapire RE (1990) The strength of weak learnability. Mach Learn 5:197–227
10. Schapire RE (2003) The boosting approach to machine learning: an overview. Nonlinear estimation and classification, pp 149–171
11. Rajadurai H, Gandhi UD (2020) A stacked ensemble learning model for intrusion detection in wireless network. Neural Comput Appl 34:1–9
12. Aguiar G, Krawczyk B, Cano A (2023) A survey on learning from imbalanced data streams: taxonomy, challenges, empirical study, and reproducible experimental framework. Mach Learn 113:1–79
13. Sleeman WC IV, Krawczyk B (2021) Multi-class imbalanced big data classification on spark. Knowl-Based Syst 212:106598
14. Shahabadi MSE, Tabrizchi H, Rafsanjani MK, Gupta B, Palmieri F (2021) A combination of clustering-based under-sampling with ensemble methods for solving imbalanced class problem in intelligent systems. Technol Forecast Soc Chang 169:120796
15. Souza MA, Cavalcanti GD, Cruz RM, Sabourin R (2019) On evaluating the online local pool generation method for imbalance learning. In: 2019 international joint conference on neural networks (IJCNN), pp 1–8. IEEE
16. Krawczyk B (2016) Learning from imbalanced data: open challenges and future directions. Prog Artif Intell 5(4):221–232
17. Xiao J, Xie L, He C, Jiang X (2012) Dynamic classifier ensemble model for customer classification with imbalanced class distribution. Expert Syst Appl 39(3):3668–3675
18. Cano A, Krawczyk B (2022) Rose: robust online self-adjusting ensemble for continual learning on imbalanced drifting data streams. Mach Learn 111(7):2561–2599
19. Elwell R, Polikar R (2011) Incremental learning of concept drift in nonstationary environments. IEEE Trans Neural Netw 22(10):1517–1531
20. Escovedo T, Da Cruz AVA, Vellasco MM, Koshiyama AS, (2013) Learning under concept drift using a neuro-evolutionary ensemble. Int J Comput Intell Appl 12(04):1340002
21. Khan I, Zhang X, Rehman M, Ali R (2020) A literature survey and empirical study of meta-learning for classifier selection. IEEE Access 8:10262–10281
22. Cruz RM, Sabourin R, Cavalcanti GD (2017) Meta-des. oracle: meta-learning and feature selection for dynamic ensemble selection. Inf Fusion 38, 84–103
23. Cruz RM, Sabourin R, Cavalcanti GD (2014) On meta-learning for dynamic ensemble selection. In: 2014 22nd international conference on pattern recognition, pp. 1230–1235. IEEE
24. Rokach L (2016) Decision forest: twenty years of research. Inf Fusion 27:111–125
25. Freund Y, Schapire RE (1995) A desicion-theoretic generalization of on-line learning and an application to boosting. In: European conference on computational learning theory, pp. 23–37. Springer
26. Hou W-H, Wang X-K, Zhang H-Y, Wang J-Q, Li L (2020) A novel dynamic ensemble selection classifier for an imbalanced data set: an application for credit risk assessment. Knowl-Based Syst 208:106462
27. Fili M, Hu G, Han C, Kort A, Trettin J, Haim H (2022) A new classification method based on dynamic ensemble selection and its application to predict variance patterns in hiv-1 env. bioRxiv, 2022–01
28. Li J, Dai C (2022) Fast prototype selection algorithm based on adjacent neighbourhood and boundary approximation. Sci Rep 12(1):1–15
29. Cruz RM, Sabourin R, Cavalcanti GD (2018) Prototype selection for dynamic classifier and ensemble selection. Neural Comput Appl 29(2):447–457
30. Cruz RM, Sabourin R, Cavalcanti GD (2017) Analyzing different prototype selection techniques for dynamic classifier and ensemble selection. In: 2017 international joint conference on neural networks (IJCNN), pp. 3959–3966. IEEE

31. Woods K, Kegelmeyer WP, Bowyer K (1997) Combination of multiple classifiers using local accuracy estimates. IEEE Trans Pattern Anal Mach Intell 19(4):405–410
32. Giacinto G, Roli F et al (2001) Dynamic classifier selection based on multiple classifier behaviour. Pattern Recogn 34(9):1879–1882
33. Didaci L, Giacinto G (2004) Dynamic classifier selection by adaptive k-nearest-neighbourhood rule. In: International workshop on multiple classifier systems, pp 174–183. Springer
34. Didaci L, Giacinto G, Roli F, Marcialis GL (2005) A study on the performances of dynamic classifier selection based on local accuracy estimation. Pattern Recogn 38(11):2188–2191
35. Zhu X, Wu X, Yang Y (2004) Dynamic classifier selection for effective mining from noisy data streams. In: Fourth IEEE international conference on data mining (ICDM'04), pp 305–312. IEEE
36. Kuncheva LI, Whitaker CJ (2003) Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy. Mach Learn 51(2):181–207
37. Ko AH, Sabourin R, Britto AS Jr (2008) From dynamic classifier selection to dynamic ensemble selection. Pattern Recogn 41(5):1718–1731
38. Sabourin M, Mitiche A, Thomas D, Nagy G (1993) Classifier combination for hand-printed digit recognition. In: Proceedings of 2nd international conference on document analysis and recognition (ICDAR'93), pp 163–166. IEEE
39. Woloszynski T, Kurzynski M, Podsiadlo P, Stachowiak GW (2012) A measure of competence based on random classification for dynamic ensemble selection. Inf Fusion 13(3):207–213
40. Woloszynski T, Kurzynski M (2011) A probabilistic model of classifier competence for dynamic ensemble selection. Pattern Recogn 44(10–11):2656–2668
41. Cavalin PR, Sabourin R, Suen CY (2013) Dynamic selection approaches for multiple classifier systems. Neural Comput Appl 22(3):673–688
42. Brun AL, Britto AS, Oliveira LS, Enembreck F, Sabourin R (2016) Contribution of data complexity features on dynamic classifier selection. In: 2016 international joint conference on neural networks (IJCNN), pp 4396–4403. IEEE
43. Ho TK, Basu M (2002) Complexity measures of supervised classification problems. IEEE Trans Pattern Anal Mach Intell 24(3):289–300
44. Dos Santos EM, Sabourin R, Maupin P (2008) A dynamic overproduce-and-choose strategy for the selection of classifier ensembles. Pattern Recogn 41(10):2993–3009
45. Monteiro M Jr, Britto AS Jr, Barddal JP, Oliveira LS, Sabourin R (2023) Exploring diversity in data complexity and classifier decision spaces for pool generation. Inf Fusion 89:567–587
46. Cruz RM, Sabourin R, Cavalcanti GD, Ren TI (2015) Meta-des: a dynamic ensemble selection framework using meta-learning. Pattern Recogn 48(5):1925–1935
47. Jain S, Shukla S, Wadhvani R (2018) Dynamic selection of normalization techniques using data complexity measures. Expert Syst Appl 106:252–262
48. Cordeiro PR, Cavalcanti GD, Cruz RM (2023) Dynamic ensemble algorithm post-selection using hardness-aware oracle. IEEE Access
49. Pinto F, Soares C, Mendes-Moreira J (2016) Chade: metalearning with classifier chains for dynamic combination of classifiers. In: Joint European conference on machine learning and knowledge discovery in databases, pp 410–425. Springer
50. Zhu X, Li J, Ren J, Wang J, Wang G (2023) Dynamic ensemble learning for multi-label classification. Inf Sci 623:94–111
51. Cruz RM, Zakane HH, Sabourin R, Cavalcanti GD (2017) Dynamic ensemble selection VS K-NN: Why and when dynamic selection obtains higher classification performance? In: 2017 seventh international conference on image processing theory, tools and applications (IPTA), pp 1–6. IEEE
52. Almeida LM, Galvao PS (2016) Ensembles with clustering-and-selection model using evolutionary algorithms. In: 2016 5th Brazilian conference on intelligent systems (BRACIS), pp 444–449. IEEE
53. Duin RP, Tax DM (2000) Experiments with classifier combining rules. In: International workshop on multiple classifier systems, pp 16–29. Springer
54. Soares RG, Santana A, Canuto AM, Souto MCP (2006) Using accuracy and diversity to select classifiers to build ensembles. In: The 2006 IEEE international joint conference on neural network proceedings, pp 1310–1316. IEEE
55. Souto MC, Soares RG, Santana A, Canuto AM (2008) Empirical comparison of dynamic classifier selection methods based on diversity and accuracy for building ensembles. In: 2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence) pp 1480–1487. IEEE
56. Oliveira D, Cavalcanti G, Sabourin R (2018) Dynamic classifier selection: recent advances and perspectives. Inf Fusion 41:195–216
57. Garcia S, Derrac J, Cano J, Herrera F (2012) Prototype selection for nearest neighbor classification: taxonomy and empirical study. IEEE Trans Pattern Anal Mach Intell 34(3):417–435
58. Triguero I, Derrac J, Garcia S, Herrera F (2012) A taxonomy and experimental study on prototype generation for nearest neighbor classification. A taxonomy and experimental study on prototype generation for nearest neighbor classification. IEEE Trans Syst Man Cybern Part C Appl Rev 42:86–100. https://doi.org/10.1109/TSMCC.2010.2103939
59. Oliveira DV, Cavalcanti GD, Sabourin R (2017) Online pruning of base classifiers for dynamic ensemble selection. Pattern Recogn 72:44–58
60. Skalak DB (1994) Prototype and feature selection by sampling and random mutation hill climbing algorithms, pp 293–301
61. Sánchez JS, Pla F, Ferri FJ (1997) Prototype selection for the nearest neighbour rule through proximity graphs. Pattern Recogn Lett 18(6):507–513
62. Wilson DL (1972) Asymptotic properties of nearest neighbor rules using edited data. IEEE Trans Syst Man Cybern 3:408–421
63. Walmsley FN, Cavalcanti GD, Sabourin R, Cruz RM (2022) An investigation into the effects of label noise on dynamic selection algorithms. Inf Fusion 80:104–120
64. Roy A, Cruz RM, Sabourin R, Cavalcanti GD (2016) Meta-regression based pool size prediction scheme for dynamic selection of classifiers. In: 2016 23rd international conference on pattern recognition (ICPR), pp 216–221. IEEE
65. Manastarla A, Silva LA (2019) A self-generating prototype method based on information entropy used for condensing data in classification tasks. In: International conference on intelligent data engineering and automated learning, pp 195–207. Springer
66. Pekalska E, Duin RP, Paclík P (2006) Prototype selection for dissimilarity-based classifiers. Pattern Recogn 39(2):189–208
67. Derrac J, Garcia S, Sanchez L, Herrera F (2015) Keel data-mining software tool: data set repository, integration of algorithms and experimental analysis framework. J Mult Valued Logic Soft Comput 17:255–287
68. Dheeru D, Taniskidou EK (2017) UCI Machine Learning Repository, http://archive.ics.uci.edu/ml, University of California, Irvine, School of Information and Computer Sciences
69. Friedman JH, Rafsky LC (1979) Multivariate generalizations of the Wald-Wolfowitz and Smirnov two-sample tests. Ann Stat, 697–717
70. Demšar J (2006) Statistical comparisons of classifiers over multiple data sets, statistical comparisons of classifiers over multiple data sets. J Mach Learn Res 7:1–30

71. Demšar J (2006) Statistical comparisons of classifiers over multiple data sets. J Mach Learn Res 7:1–30
72. Meyes R, Lu M, Puiseau CW, Meisen T (2019) Ablation studies in artificial neural networks. arXiv preprint arXiv:1901.08644
73. Gini C (1912) Variabilità e mutabilità
74. Fernández-Delgado M, Cernadas E, Barro S, Amorim D (2014) Do we need hundreds of classifiers to solve real world classification problems? J Mach Learn Res 15(1):3133–3181
75. Cruz RM, Sabourin R, Cavalcanti GD (2015) A deep analysis of the meta-des framework for dynamic selection of ensemble of classifiers. arXiv preprint arXiv:1509.00825
76. Fränti P, Sieranoja S (2018) K-means properties on six clustering benchmark datasets . http://cs.uef.fi/sipu/datasets/
77. Lorena AC, Garcia LP, Lehmann J, Souto MC, Ho TK (2019) How complex is your classification problem? A survey on measuring classification complexity. ACM Computi Surv (CSUR) 52(5):1–34

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.